

Name: Hope L Chibondwe

Reg Number:H250404G

Course: Object Oriented Programming

Department: Computer Science

Lab report: labs 1-10

LAB 1: Classes and Objects

Objectives

1) To define a class with fields and methods.
2) To create objects with new and use the dot operator.
3) To understand that object variables hold references, not the objects themselves.
4) To recognize null and avoid NullPointerException.

Sample programs

Student.java

```
public class Student {
    //EXPLAIN: instance variables define the STATE of every Student object
    String name;
    int rollNo;
    double gpa;

    //Explain: instance methods define BEHAVIOUR; they can read and change the
fields
    void display() {
        System.out.println("Name: " + name + ", Roll No: " + rollNo + ", GPA: " +
gpa);
    }

    boolean isHonours() {
        return gpa >= 3.5; //EXPLAIN: returns true/false based on the object's
own gpa

    }

        public static void main(String[] args) {
        // EXPLAIN: 'new' allocates memory on the heap and returns a reference
        Student s1 = new Student();
        s1.name = "Tariro";
        s1.rollNo = 101;
        s1.gpa = 3.8;

        Student s2 = new Student();
        s2.name = "Farai";
        s2.rollNo = 102;
        s2.gpa = 3.1;
```

```
        s1.display();
        s2.display();
        System.out.println(s1.name + " honours? " + s1.isHonours());
        System.out.println(s2.name + " honours? " + s2.isHonours());
    }
}
```

Student.java: a class with fields and methods, and two objects made from it.

Output:

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab1> javac Student.java
PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab1> java Student
Name: Tariro, Roll No: 101, GPA: 3.8
Name: Farai, Roll No: 102, GPA: 3.1
Tariro honours? true
Farai honours? false
PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab1>

ReferenceDemo.java

```
public class ReferenceDemo {
    public static void main(String[] args) {
        Student a = new Student();
        a.name = "Original";

        // EXPLAIN: b copies the REFERENCE, not the object -> a and b are aliases
        Student b = a;
        b.name = "Changed via b";
        System.out.println(a.name);           // prints: Changed via b

        // EXPLAIN: == on references compares addresses, not contents
        Student c = new Student();
        c.name = "Changed via b";
        System.out.println(a == b);           // true  (same object)
        System.out.println(a == c);           // false (different objects)

        // EXPLAIN: a reference that points to nothing is null
        Student d = null;
        if (d == null) {
            System.out.println("d does not refer to any object");
        }
        // d.display();  // would throw NullPointerException at runtime
```

```
    }
}
```

Student b = a, copies the reference, so a and b are aliases. == compares references. A null reference points to nothing.

Output:

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab1> javac ReferenceDemo.java

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab1> java ReferenceDemo

Changed via b

true

false

d does not refer to any object

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab1>

Course.java

```
public class Course {
    String title;
    Student topStudent;   // EXPLAIN: a field can itself be a reference to
another object

    public static void main(String[] args) {
        Student s = new Student();
        s.name = "Rudo";
        s.gpa = 3.9;

        Course c = new Course();
        c.title = "Object-Oriented Programming";
        c.topStudent = s;

        // EXPLAIN: chained dot operator: course -> student -> name
        System.out.println(c.title + " top student: " + c.topStudent.name);
    }
}
```

An object holding another object, accessed with a chained dot. (c.topStudent.name).

Output:

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab1> javac Course.java

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab1> java Course

Object-Oriented Programming top student: Rudo

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab1>

Exercises

Car.java

```
public class Car {

    // fields (state)
    String brand;
    int year;
    int mileage;

    // method (behaviour)
    void display() {
        System.out.println("Brand: " + brand + ", Year: " + year + ", Mileage: "
+ mileage);
    }

    // Exercise 2
    boolean isAntique() {
        // hint: a car is antique if (2026 - year) is more than 25
        return (2026 - year) > 25;
    }

    public static void main(String[] args) {
        Car c1 = new Car();
        c1.brand = "Toyota Corolla";
        c1.year = 2010;
        c1.mileage = 150000;
```

```
        Car c2 = new Car();
        c2.brand = "Honda Fit";
        c2.year = 2020;
        c2.mileage = 50000;

        Car c3 = new Car();
        c3.brand = "Mazda 6";
        c3.year = 1998;
        c3.mileage = 200000;

        c1.display();
        c2.display();
        c3.display();
        System.out.println("Is c1 antique? " + c1.isAntique());
        System.out.println("Is c2 antique? " + c2.isAntique());
        System.out.println("Is c3 antique? " + c3.isAntique());
    }
}
```

1) **isAntique().** It returns true when 2026 - year is more than 25. Output: c1 false, c2 false, c3 true.
2) **Rectangle array.** Paste Rectangle.java. The areas are 20, 6, 24, 21 and 25, so the largest is 5.0 x 5.0 (area 25.0, perimeter 20.0).
3) r1 holds a reference (an address) to the rectangle, not the rectangle itself. The line Rectangle r2 = r1, copies that address, so r1 and r2 both point to the same single object. Changing r2.width therefore also changes what r1 sees.

r2 = r1 Prediction:
```
// The lines that will run are:

// Car c1 = new Car();
// c1.brand = "Toyota Corolla";
// c1.year = 2010;
// c1.mileage = 150000;
//
// Car c2 = new Car();
// c2.brand = "Honda Fit";
// c2.year = 2020;
// c2.mileage = 50000;
//
// Car c3 = new Car();
// c3.brand = "Mazda 6";
// c3.year = 1998;
// c3.mileage = 200000;
```

Actual Output:

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab1> javac Car.java

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab1> java Car

Brand: Toyota Corolla, Year: 2010, Mileage: 150000

Brand: Honda Fit, Year: 2020, Mileage: 50000

Brand: Mazda 6, Year: 1998, Mileage: 200000

Is c1 antique? false

Is c2 antique? false

Is c3 antique? true

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab1>

Person.java

```java
public class Person {
    String name;
    Address address;     // a field that holds a reference to another object

    public static void main(String[] args) {
        // Person WITH an address
        Address a1 = new Address();
        a1.street = "12 Samora Machel Ave";
        a1.city = "Harare";

        Person p1 = new Person();
        p1.name = "Tendai";
        p1.address = a1;

        // chained dot: person -> address -> city
        System.out.println(p1.name + " lives in " + p1.address.city);

        // Person WITHOUT an address
        Person p2 = new Person();
        p2.name = "Chipo";
        // p2.address was never set, so it is null

        // Step A: check for null first (safe)
        if (p2.address == null) {
```

```
            System.out.println(p2.name + " has no address yet");
        } else {
            System.out.println(p2.name + " lives in " + p2.address.city);
        }

        // Step B: this will crash because p2.address is null
        //System.out.println(p2.address.city);



    }
}
```

Output:

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab1> javac Person.java

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab1> java Person

Tendai lives in Harare

Chipo has no address yet

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab1>

Explanation:

p2.address is null because an object field has no object until one is assigned. Reading .city from null throws NullPointerException. Checking if (p2.address == null) first prevents the crash.

Conclusion:

A class is a blueprint and an object is one instance of it. Object variables hold references, so assigning one variable to another creates an alias, not a copy. Using a null reference causes a NullPointerException, so it must be checked.

Viva answers

1) A class is a blueprint that describes the fields and methods an entity has whilst an object is one concrete instance of that class, created with new.
2) Objects live on the heap, and a variable such as Student s1 holds a reference to the object on the heap.

LAB2: Constructors, the this Keyword and static Members

Objectives

1) To initialize objects with constructors and to overload and chain them with this(...).
2) To use this to tell fields apart from parameters.
3) To explain the difference between instance and static members.
4) To use static and instance initializer blocks.

Sample programs

Book.java

```
public class Book {

    // EXPLAIN: static field -> ONE copy shared by all Book objects
    private static int count = 0;

    // instance fields -> each Book has its own copy
    private final int id;
    private String title;
    private String author;
    private double price;

    // EXPLAIN: static initialiser block runs once, when the class is first
loaded
    static {
        System.out.println("Book class loaded");
    }

    // EXPLAIN: instance initialiser block runs before EVERY constructor body
    {
        count++;
        id = count;   // final field assigned exactly once
    }
```

```
    // Constructor 1: full initialisation
    public Book(String title, String author, double price) {
        this.title = title;       // EXPLAIN: this.title = field, title =
parameter
        this.author = author;
        this.price = price;
    }

    // Constructor 2: constructor chaining with this(...)
    public Book(String title, String author) {
        this(title, author, 0.0);  // Makes one constructor reuse another
    }

    // Constructor 3: no-argument constructor
    public Book() {
        this("Untitled", "Unknown");
    }

    // EXPLAIN: static method -> no 'this'; it can only use static members
directly
    public static int getCount() {
        return count;
    }

    public void display() {
        System.out.printf("#%d %s by %s ($%.2f)%n", id, title, author, price);
    }

    public static void main(String[] args) {
        Book b1 = new Book("Clean Code", "Robert Martin", 39.99);
        Book b2 = new Book("Effective Java", "Joshua Bloch");
        Book b3 = new Book();

        b1.display();
        b2.display();
        b3.display();
        System.out.println("Total books created: " + Book.getCount());
    }
}
```

Output:

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab2> javac Book.java

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab2> java Book.java

Book class loaded

#1 Clean Code by Robert Martin ($39.99)

#2 Effective Java by Joshua Bloch ($0.00)

#3 Untitled by Unknown ($0.00)

Total books created: 3

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab2>

Explanation:

three constructors, with chaining through this(...). A static counter is shared by all books.

InitOrder.java

```java
public class InitOrder {
    static int a = show("static field");
    int b = show("instance field");

    static { show("static block"); }
    { show("instance block"); }

    InitOrder() { show("constructor"); }

    static int show(String msg) {
        System.out.println(msg);
        return 1;
    }

    public static void main(String[] args) {
        System.out.println("--- first object ---");
        new InitOrder();
        System.out.println("--- second object ---");
        new InitOrder();
    }
}
```

Static parts run once when the class loads. Instance parts run for every object, before the constructor body.

Output:

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab2> javac InitOrder.java

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab2> java InitOrder

static field

static block

--- first object ---

instance field

instance block

constructor

--- second object ---

instance field

instance block

constructor

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab2>

Point.java

```
public class Point {
    private final int x, y;

    public Point(int x, int y) { this.x = x; this.y = y; }

    // EXPLAIN: copy constructor creates an independent object with the same
values
    public Point(Point other) { this(other.x, other.y); }

    public String toString() { return "(" + x + ", " + y + ")"; }

    public static void main(String[] args) {
        Point p1 = new Point(3, 4);
```

```
        Point p2 = new Point(p1);   // a true copy, not an alias
        System.out.println(p1 + " " + p2 + " same object? " + (p1 == p2));
    }
}
```

(copy constructor): new Point(p1) makes an independent copy. Output: (3, 4) (3, 4) same object? False.

Output:

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab2> javac Point.java

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab2> java Point

(3, 4) (3, 4) same object? false

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab2>

Exercises

Circle.java

```
public class Circle {

    // static: ONE shared copy for all circles (Exercise 2)
    private static int count = 0;

    // instance field: each circle has its own radius
    private double radius;

    // Exercise 1: constructor taking a radius
    public Circle(double radius) {
        this.radius = radius;     // field on the left, parameter on the right
        count++;            // a new circle exists
    }

    // Exercise 1: no-argument constructor, radius defaults to 1.0
    public Circle() {
        this(1.0);          // reuse the constructor above
    }

    // Exercise 2: static getter
    public static int getCount() {
        return count;
```

```
    }

    public double area() {
        return Math.PI * radius * radius;
    }

    public static void main(String[] args) {
        Circle c1 = new Circle(5.0);
        Circle c2 = new Circle();
        Circle c3 = new Circle(2.5);

        System.out.println("c1 area: " + c1.area());
        System.out.println("c2 area: " + c2.area());
        System.out.println("c3 area: " + c3.area());
        System.out.println("Circles created: " + Circle.getCount());
    }
}
```

Output:

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab2> javac Circle.java

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab2> java Circle

c1 area: 78.53981633974483

c2 area: 3.141592653589793

c3 area: 19.634954084936208

Circles created: 3

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab2>

Explanation

Circle with two constructors - The no-argument constructor calls this(1.0).

Static counter. count is static and getCount() is a static getter.

Employee.java

```java
public class Employee {

    private static int nextId = 1;        // shared counter

    private final int employeeId;
    private String name;
    private String department;
    private double salary;

    // Constructor 1: full
    public Employee(String name, String department, double salary) {
        this.employeeId = nextId++;       // use the current number, then add 1
        this.name = name;
        this.department = department;
        this.salary = salary;
    }

    // Constructor 2: name and department only, salary defaults to 0
    public Employee(String name, String department) {
        this(name, department, 0.0);   // reuse the constructor above
    }

    // Constructor 3: no arguments
    public Employee() {
        this("Unknown", "Unassigned");
    }

    // Copy constructor: same details, but a NEW id
    public Employee(Employee other) {
        this(other.name, other.department, other.salary);
    }

    public void display() {
        System.out.println("#" + employeeId + " " + name + ", " + department + ",
$" + salary);
    }

    public static void main(String[] args) {
        Employee e1 = new Employee("Tendai", "IT", 1500);
        Employee e2 = new Employee("Chipo", "HR");
        Employee e3 = new Employee();
        Employee e4 = new Employee(e1);      // copy of e1

        e1.display();
        e2.display();
```

```
        e3.display();
        e4.display();
    }
}
```

Output:

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab2> javac Employee.java

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab2> java Employee

#1 Tendai, IT, $1500.0

#2 Chipo, HR, $0.0

#3 Unknown, Unassigned, $0.0

#4 Tendai, IT, $1500.0

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab2>

Explanation

employeeId is final because an id is set once. nextId is static because there must be one shared counter. The copy #4 has the same details as #1 but a new id.

BlockOrder.java

```
public class BlockOrder {

    static {
        System.out.println("static block 1");
    }

    {
        System.out.println("instance block 1");
    }

    BlockOrder() {
        System.out.println("constructor");
    }

    static {
        System.out.println("static block 2");
```

```
    }

    {
        System.out.println("instance block 2");
    }
    //runs static blocks first, then instance blocks, then constructor. Static
blocks run only once, when the class is loaded. Instance blocks run every time an
object is created, before the constructor.
    public static void main(String[] args) {
        System.out.println("--- main starts ---");
        new BlockOrder();
        System.out.println("--- second object ---");
        new BlockOrder();
    }
}
// Prediction - The lines that will run are: Static block 1, Static block 2, ---
main starts ---, instance block 1, instance block 2, constructor, --- second
object ---, instance block 1, instance block 2, constructor.
```

Output:

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab2> javac BlockOrder.java

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab2> java BlockOrder

static block 1

static block 2

--- main starts ---

instance block 1

instance block 2

constructor

--- second object ---

instance block 1

instance block 2

constructor

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab2>

Explanation

Static blocks run once, when the class is loaded, which is before main starts, so they don’t run again for the second object. Instance blocks run every time an object is created, in the order written, before the constructor body. The constructor is written between the instance blocks in the file but still runs after both.

Temperature.java

```
public class Temperature {

    private final double celsius;     // store one unit internally

    // private constructor: only code inside this class can call it
    private Temperature(double celsius) {
        this.celsius = celsius;
    }

    // static factory method 1
    public static Temperature fromCelsius(double c) {
        return new Temperature(c);
    }

    // static factory method 2: convert F to C first
    // formula: C = (F - 32) * 5 / 9
    public static Temperature fromFahrenheit(double f) {
        return new Temperature((f - 32) * 5 / 9);
    }

    public double getCelsius() {
        return celsius;
    }

    public double getFahrenheit() {
        return celsius * 9 / 5 + 32;      // formula: F = C * 9 / 5 + 32
    }
    //“fromCelsius(25) and fromFahrenheit(25) both take one double, so two
constructors would have the same signature, which Java doesn't allow. A factory
method has its own descriptive name, so the reader knows what the number means.”

    public static void main(String[] args) {
```

```
        Temperature t1 = Temperature.fromCelsius(25);
        Temperature t2 = Temperature.fromFahrenheit(98.6);

        System.out.println("t1: " + t1.getCelsius() + " C = " +
t1.getFahrenheit() + " F");
        System.out.println("t2: " + t2.getCelsius() + " C = " +
t2.getFahrenheit() + " F");

        // new Temperature(30);   // compile error if called from another class:
Temperature(double) has private access

    }
}
```

Output:

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab2> Javac Temperature.java

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab2> java Temperature

t1: 25.0 C = 77.0 F

t2: 37.0 C = 98.6 F

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab2>

Testing TemperatureTest.java

```
new Temperature(30);   // compile error if called from another class:
Temperature(double) has private access
```

Explanation

fromCelsius(25) and fromFahrenheit(25) both take one double, so two constructors would have the same signature, which Java does not allow. A factory method has its own name, so the reader knows what the number means. new Temperature(98.6) doesn't say.

Conclusion

Constructors set up an object safely when it is created, and chaining with this(...) avoids repeating code. Static members belong to the class and are shared, while

instance members belong to each object. Private constructors with static factory methods give clearer names and control over how objects are created.

Viva questions

1) What happens to the default constructor when you write your own? It disappears, so if you still need a no-argument constructor you must write it.
2) What does this mean, and why must this(...) come first? this is a reference to the current object. A constructor must finish its setup in order, so a call to another constructor comes before anything else.
3) What is the difference between a static and an instance member? A static member has one copy shared by the whole class, so you can use it without creating an object. An instance member has a separate copy in each object.

LAB 3: Encapsulation and Access Modifiers

Objectives

1) To hide data with private fields and expose it through controlled methods.
2) To apply the four access levels (private, default, protected, public).
3) To enforce validation rules in setters and constructors.
4) To design classes that cannot be put into an invalid state.

Sample programs

BankAccount.java

```
import java.util.ArrayList;
import java.util.List;

public class BankAccount {

    // EXPLAIN: private -> no code outside this class can touch these directly
    private final String accountNumber;
    private String ownerName;
    private double balance;
    private final List<String> history = new ArrayList<>();

    public BankAccount(String accountNumber, String ownerName, double
openingBalance) {
        if (openingBalance < 0) {
```

```
            throw new IllegalArgumentException("Opening balance cannot be
negative");
        }
        this.accountNumber = accountNumber;
        setOwnerName(ownerName);          // reuse validation in the setter
        this.balance = openingBalance;
    }

    // Getters: read-only access
    public String getAccountNumber() { return accountNumber; }
    public String getOwnerName()     { return ownerName; }
    public double getBalance()       { return balance; }

    // Setter with validation
    public void setOwnerName(String ownerName) {
        if (ownerName == null || ownerName.isBlank()) {
            throw new IllegalArgumentException("Owner name required");
        }
        this.ownerName = ownerName.trim();
    }

    // EXPLAIN: no setBalance() on purpose -> balance changes only via
deposit/withdraw
    public void deposit(double amount) {
        if (amount <= 0) {
            throw new IllegalArgumentException("Deposit must be positive");
        }
        balance += amount;
        history.add("Deposit: " + amount);
    }

    public boolean withdraw(double amount) {
        if (amount <= 0) {
            throw new IllegalArgumentException("Withdrawal must be positive");
        }
        if (amount > balance) {
            return false;                 // insufficient funds, state unchanged
        }
        balance -= amount;
        history.add("Withdrawal: " + amount);
        return true;
    }

    public List<String> getHistory() {
    return List.copyOf(history);    // makes an unmodifiable copy
```

```
}

    @Override
    public String toString() {
        return String.format("%s [%s] balance: %.2f", accountNumber, ownerName,
balance);
    }
}
```

BankDemo.java

```
public class BankDemo {
    public static void main(String[] args) {
        BankAccount acc = new BankAccount("ACC-001", "Tendai", 500.00);

        acc.deposit(250.00);
        System.out.println(acc);

        System.out.println("Withdraw 100: " + acc.withdraw(100.00));
        System.out.println("Withdraw 5000: " + acc.withdraw(5000.00));
        System.out.println(acc);

        // acc.balance = 1_000_000;   // COMPILE ERROR: balance has private
access

        try {
            acc.deposit(-50);
        } catch (IllegalArgumentException e) {
            System.out.println("Rejected: " + e.getMessage());
        }
    }
}
```

The fields are private, and there is no setBalance() on purpose, so the balance only changes through deposit and withdraw, which check the amount first.

Output:

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab3> javac *.java

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab3> java BankDemo

ACC-001 [Tendai] balance: 750.00

Withdraw 100: true

Withdraw 5000: false

ACC-001 [Tendai] balance: 650.00

Rejected: Deposit must be positive

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab3>

Team.java

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class Team {
    private final List<String> members = new ArrayList<>();

    public void addMember(String name) { members.add(name); }

    // BAD: caller receives the real list and can modify it directly
    public List<String> getMembersUnsafe() { return members; }

    // GOOD: caller receives a read-only view
    public List<String> getMembers() { return
Collections.unmodifiableList(members); }

    public static void main(String[] args) {
        Team t = new Team();
        t.addMember("Chipo");

        t.getMembersUnsafe().clear();                // silently empties the
team!
        System.out.println("After unsafe clear: " + t.getMembers().size());

        t.addMember("Chipo");
        try {
            t.getMembers().clear();                  // blocked
        } catch (UnsupportedOperationException e) {
            System.out.println("Safe getter blocked modification");
        }
```

```
    }
}
```

The defensive-copy trap: a getter that returns the real list lets outside code empty it. A getter that returns an unmodifiable view blocks that.

Output:

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab3> javac Team.java

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab3> java Team

After unsafe clear: 0

Safe getter blocked modification

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab3>

Exercises

Student.java

```
public class Student {

    // private: no outside code can touch these directly
    private String name;
    private int marks;

    // the constructor reuses the setters, so validation is written once
    public Student(String name, int marks) {
        this.name = name;
        setMarks(marks);
    }

    public String getName() { return name; }
    public int getMarks()   { return marks; }

    // Exercise 1: reject invalid marks
    public void setMarks(int marks) {
        if (marks < 0 || marks > 100) {
            throw new IllegalArgumentException("Marks must be between 0 and
100");
```

```
        }
        this.marks = marks;
    }

    // Exercise 2: grade is worked out from marks, NOT stored in a field
    // I chose A at 75+, B at 60+ and C at 50+, F below that
    public String getGrade() {
        if (marks >= 75) {
            return "A";
        } else if (marks >= 60) {
            return "B";
        } else if (marks >= 50) {
            return "C";
        } else {
            return "F";
        }
    }

    public static void main(String[] args) {
        Student s1 = new Student( "Tariro",  82);
        System.out.println(s1.getName() + ": " + s1.getMarks() + " -> " +
s1.getGrade());

        s1.setMarks( 58);
        System.out.println(s1.getName() + ": " + s1.getMarks() + " -> " +
s1.getGrade());

        // invalid marks must be rejected
        try {
            s1.setMarks( 150);
        } catch (IllegalArgumentException e) {
            System.out.println("Rejected: " + e.getMessage());
        }
    }
}
```

Output:

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab3> javac Student.java

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab3> java Student

Tariro: 82 -> A

Tariro: 58 -> C

Rejected: Marks must be between 0 and 100

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab3>

Explanation

There is no grade field because it would go out of date whenever setMarks changes the marks. Working it out from marks each time means it can never be wrong.

HistoryDemo.java

```
import java.util.List;

public class HistoryDemo {
    public static void main(String[] args) {
        BankAccount acc = new BankAccount("ACC-002", "Chipo", 100.00);
        acc.deposit(50);
        acc.withdraw(30);
        acc.withdraw(9999);      // fails, should NOT appear in history

        List<String> h = acc.getHistory();
        System.out.println(h);

        try {
            h.add("Deposit: 1000000");    // try to cheat
        } catch (UnsupportedOperationException e) {
            System.out.println("History cannot be modified from outside");
        }
    }
}
```

Output

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab3> javac *.java

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab3> java HistoryDemo

[Deposit: 50.0, Withdrawal: 30.0]

History cannot be modified from outside

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab3>

Explanation

The failed withdraw(9999) is not in the history because it returns before reaching the history.add line. getHistory() returns a copy, so outside code cannot change the real list. Returning history itself would let any caller add or delete transactions.

ImmutablePerson.java

```java
public final class ImmutablePerson {      // choice 1

    private final String name;            // choice 2
    private final int age;

    public ImmutablePerson(String name, int age) {
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("Name required");
        }
        if (age < 0) {
            throw new IllegalArgumentException("Age cannot be negative");
        }
        this.name = name;
        this.age = age;
    }

    // getters only: choice 3 (no setters)
    public String getName() { return name; }
    public int getAge()     { return age; }

    // "changing" a person creates a NEW object instead
    public ImmutablePerson withAge(int newAge) {
        return new ImmutablePerson(name, newAge);
    }

    @Override
```

```
    public String toString() {
        return name + " (" + age + ")";
    }

    public static void main(String[] args) {
        ImmutablePerson p1 = new ImmutablePerson("Tendai", 20);
        ImmutablePerson p2 = p1.withAge(21);

        System.out.println(p1);    // still 20
        System.out.println(p2);    // new object with 21

    }
}
```

Output:

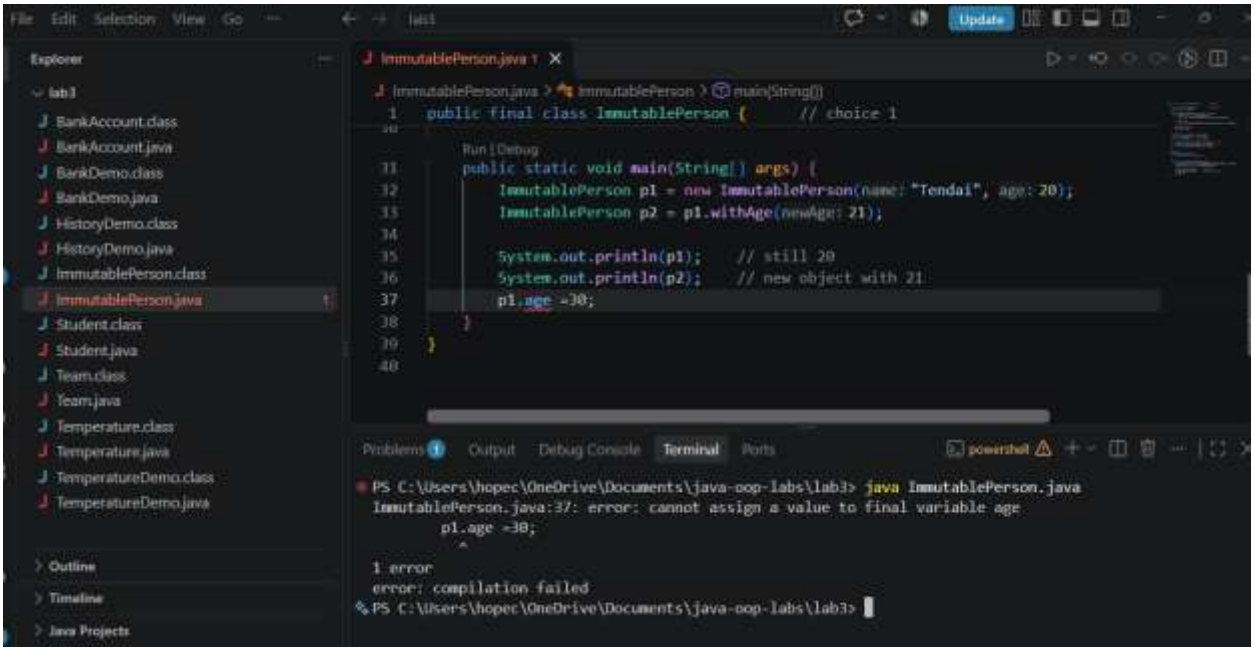
PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab3> javac ImmutablePerson.java

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab3> java ImmutablePerson

Tendai (20)

Tendai (21)

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab3>



Age has private access, and cannot assign a value to a final variable.

Explanation

1) final class: nobody can subclass it, so no subclass can add a setter or override a getter to make it changeable.
2) private final fields: private stops outside code touching them, and final means they are assigned once in the constructor and never again.
3) No setters: a setter would allow the object to change after it is created.
4) withAge returns a new object: a “change” produces a new object, so p1 is still 20 afterwards.

Temperature.java

```
public class Temperature {

    // the ONLY place the unit is decided: kelvin
    private final double kelvin;

    // private constructor: validation is written once, here
    private Temperature(double kelvin) {
        if (kelvin < 0) {      // absolute zero
            throw new IllegalArgumentException("Temperature below absolute
zero");
        }
        this.kelvin = kelvin;
```

```
    }

    // factories convert whatever the caller has into kelvin
    public static Temperature fromKelvin(double k) {
        return new Temperature(k);
    }

    public static Temperature fromCelsius(double c) {
        return new Temperature(c + 273.15);
    }

    public static Temperature fromFahrenheit(double f) {
        // formula: C = (F - 32) * 5 / 9, then K = C + 273.15
        return new Temperature((f - 32) * 5 / 9 + 273.15);
    }

    // getters convert kelvin back out
    public double getKelvin() {
        return kelvin;
    }

    public double getCelsius() {
        return kelvin - 273.15;
    }

    public double getFahrenheit() {
        return getCelsius() * 9 / 5 + 32;
    }
}
```

TemperatureDemo.java

```
public class TemperatureDemo {
    public static void main(String[] args) {
        Temperature t1 = Temperature.fromCelsius(25);
        Temperature t2 = Temperature.fromFahrenheit(212);
        Temperature t3 = Temperature.fromKelvin(0);

        System.out.printf("t1: %.2f C, %.2f F%n", t1.getCelsius(),
t1.getFahrenheit());
        System.out.printf("t2: %.2f C, %.2f F%n", t2.getCelsius(),
t2.getFahrenheit());
        System.out.printf("t3: %.2f C, %.2f F%n", t3.getCelsius(),
t3.getFahrenheit());
```

```
        try {
            Temperature.fromCelsius(-300);
        } catch (IllegalArgumentException e) {
            System.out.println("Rejected: " + e.getMessage());
        }
    }
}
```

Output:

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab3> javac *.java

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab3> java TemperatureDemo

t1: 25.00 C, 77.00 F

t2: 100.00 C, 212.00 F

t3: -273.15 C, -459.67 F

Rejected: Temperature below absolute zero

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab3>

Output of TemperatureDemo.java after changing the inside of Temperature to store Celcius

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab3> javac *.java

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab3> java TemperatureDemo

t1: 25.00 C, 77.00 F

t2: 100.00 C, 212.00 F

t3: -273.15 C, -459.67 F

Rejected: Temperature below absolute zero

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab3>

Explanation

The field is private, so the caller never depended on it. The caller only used the public methods, which are the class's contract. Encapsulation means the internal unit can change without breaking any code that uses the class.

Conclusion

Encapsulation hides a class's data and lets it change only through methods that validate. This keeps an object always in a valid state, and it lets the internals change without breaking callers. Getters must not return references to mutable internal objects, or the hiding leaks.

Questions

1) What are the four access levels? private is the same class only. Default is the same package. protected is the same package plus subclasses. public is anywhere.
2) Why make fields private? So no code outside can put the object into an invalid state, and so the internals can change later.
3) What does final mean on a variable, a method and a class? A final variable can be assigned only once. A final method cannot be overridden. A final class cannot be extended.
4) Why is returning an internal list from a getter dangerous? The caller gets the real list and can change it directly, so the validation is bypassed.

LAB 4: Inheritance

Objectives

1) Create subclasses with extends and reuse superclass code.
2) Call superclass constructors and methods with super.
3) Use protected, final and @Override correctly.
4) Explain constructor call order and decide between inheritance (is-a) and composition (has-a).

Sample programs

Employee.java

```
public class Employee {

    private final String name;
    protected double baseSalary;      // EXPLAIN: protected -> visible to
subclasses

    public Employee(String name, double baseSalary) {
        this.name = name;
        this.baseSalary = baseSalary;
        System.out.println("Employee constructor: " + name);
    }

    public String getName() { return name; }

    public double calculatePay() {
        return baseSalary;
    }

    public void describe() {
        System.out.printf("%s earns %.2f%n", name, calculatePay());
    }
}
```

Manager.java

```
public class Manager extends Employee {     // EXPLAIN: Manager IS-A Employee

    private double bonus;

    public Manager(String name, double baseSalary, double bonus) {
        super(name, baseSalary);            // EXPLAIN: must be first; runs
Employee's constructor
        this.bonus = bonus;
        System.out.println("Manager constructor: " + name);
    }

    @Override                               // EXPLAIN: compiler checks we really
override
    public double calculatePay() {
        return super.calculatePay() + bonus; // reuse the parent's logic, then
extend it
```

```
    }
}
```

CompanyDemo.java

```
public class CompanyDemo {
public static void main(String[] args) {
Employee e = new Employee("Kudzai", 2000);
Manager m = new Manager("Nyasha", 3500, 800);
Developer d = new Developer("Tatenda", 3000, 10);
System.out.println("----");
e.describe();
m.describe(); // describe() is inherited; calculatePay() is Manager's version

d.describe();
}
}
```

Output:

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab4> javac *.java

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab4> java CompanyDemo

Employee constructor: Kudzai

Employee constructor: Nyasha

Manager constructor: Nyasha

Employee constructor: Tatenda

Kudzai earns 2000.00

Nyasha earns 4300.00

Tatenda earns 3250.00

Chain.java

```
class A {
A() { System.out.println("A constructor"); }
}
class B extends A {
B() { System.out.println("B constructor"); } // implicit super() -> A()
}
class C extends B {
C() { System.out.println("C constructor"); } // implicit super() -> B()
}
public class Chain {
public static void main(String[] args) {
new C();
}
}
```

constructors run from the top of the chain down: A constructor, B constructor, C constructor.

CompositionDemo.java

```
public class CompanyDemo {
public static void main(String[] args) {
Employee e = new Employee("Kudzai", 2000);
Manager m = new Manager("Nyasha", 3500, 800);
Developer d = new Developer("Tatenda", 3000, 10);
System.out.println("----");
e.describe();
m.describe(); // describe() is inherited; calculatePay() is Manager's version

d.describe();
}
}
```

a Car has an Engine (has-a), and the Car class is final.

Output:

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab4> javac *.java

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab4> java CompositionDemo

Engine started

Car is moving

Car with an engine

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab4>

Exercises

Person.java

```java
public class Person {

    private String name;
    private int age;

    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public String getName() { return name; }
    public int getAge()     { return age; }

    // inherited by every subclass
    public void display() {
        System.out.println("Name: " + name + ", Age: " + age);
    }
}
```

Student.java

```java
public class Student extends Person {

    private String course;

    public Student(String name, int age, String course) {
        super(name,age);        // pass name and age up to Person
        this.course = course;
    }

    public String getCourse() { return course; }
}
```

Teacher.java

```java
public class Teacher extends Person {
    private String subject;

    public Teacher(String name, int age, String subject) {
        super(name,age);        // pass name and age up to Person
        this.subject = subject;
    }

    public String getSubject() { return subject; }
}
```

PersonDemo.java

```java
public class PersonDemo {
    public static void main(String[] args) {
        Person p = new Person("Tendai", 40);
        Student s = new Student("Tariro", 20, "Computer Science");
        Teacher t = new Teacher("Mr Moyo", 45, "Mathematics");

        p.display();
        s.display();     // inherited, Student never wrote it
        System.out.println("Course: " + s.getCourse());
        t.display();     // inherited, Teacher never wrote it
        System.out.println("Subject: " + t.getSubject());
    }
}
```

Output:

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab4> cd exercises

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab4\exercises> javac *.java

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab4\exercises> java PersonDemo

Name: Tendai, Age: 40

Name: Tariro, Age: 20

Course: Computer Science

Name: Mr Moyo, Age: 45

Subject: Mathematics

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab4\exercises>

display() is written only in Person, and both subclasses inherit it.

Explanation: Student and Teacher both extend Person, so name, age and display() are written once and shared. Each subclass only adds what is different (course or subject).

Vehicle, Car, ElectricCar (multilevel).

Vehicle.java

```
public class Vehicle {

    private String brand;

    public Vehicle(String brand) {
        this.brand = brand;
        System.out.println("Vehicle constructor: " + brand);
    }

    public String getBrand() { return brand; }
}
```

Car.java

```
public class Car extends Vehicle {

    private int doors;

    public Car(String brand, int doors) {
        super(brand);                  // pass brand up to Vehicle
        this.doors = doors;
        System.out.println("Car constructor: " + doors + " doors");
    }

    public int getDoors() { return doors; }
}
```

ElectricCar.java

```
public class ElectricCar extends Car {

    private int batteryKwh;

    public ElectricCar(String brand, int doors, int batteryKwh) {
        super(brand, doors);        // pass brand and doors up to Car, which
passes brand up to Vehicle
        this.batteryKwh = batteryKwh;
        System.out.println("ElectricCar constructor: " + batteryKwh + " kWh");
    }

    public int getBatteryKwh() { return batteryKwh; }
}
```

VehicleDemo.java

```
public class VehicleDemo {
    public static void main(String[] args) {
        System.out.println("Creating an ElectricCar...");
        ElectricCar e = new ElectricCar("Nissan Leaf", 5, 40);

        System.out.println("---");
        System.out.println("Brand: " + e.getBrand());      // from Vehicle
        System.out.println("Doors: " + e.getDoors());      // from Car
        System.out.println("Battery: " + e.getBatteryKwh() + " kWh");  // from
ElectricCar
    }
}
```

Output:

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab4\exercises> javac *.java

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab4\exercises> java VehicleDemo

Creating an ElectricCar...

Vehicle constructor: Nissan Leaf

Car constructor: 5 doors

ElectricCar constructor: 40 kWh

Brand: Nissan Leaf

Doors: 5

Battery: 40 kWh

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab4\exercises>

Explanation: new ElectricCar calls super(brand, doors), which jumps to Car. Car calls super(brand), which jumps to Vehicle. Vehicle finishes first, then Car, then ElectricCar.

Animal.java

```
public class Animal {

    private String name;

    // the ONLY constructor: it needs a name
    public Animal(String name) {
        this.name = name;
    }

    public String getName() { return name; }
}
```

Dog.java

```
public class Dog extends Animal {

    public Dog(String name) {
        super(name);                // calls Animal(String name)
        System.out.println("Dog created: " + name);
    }
}
```

DogDemo.java

```java
public class DogDemo {
    public static void main(String[] args) {
        Dog d = new Dog("Rex");
        System.out.println("Name: " + d.getName());
    }
}
```

Output:

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab4\exercises> cd ..

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab4> cd ex4

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab4\ex4> javac *.java

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab4\ex4> java DogDemo

Dog created: Rex

Name: Rex

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab4\ex4>

Stack extends ArrayList versus Stack that contains an ArrayList.

MyStack.java

```java
import java.util.ArrayList;

public class MyStack {

    // HAS-A: the list is hidden inside
    private final ArrayList<String> items = new ArrayList<>();

    public void push(String item) {
        items.add(item);
    }

    public String pop() {
        if (items.isEmpty()) {
            throw new IllegalStateException("Stack is empty");
        }
```

```
        return items.remove(items.size() - 1);
    }

    public String peek() {
        if (items.isEmpty()) {
            throw new IllegalStateException("Stack is empty");
        }
        return items.get(items.size() - 1);
    }

    public boolean isEmpty() {
        return items.isEmpty();
    }
}
```

BadStack.java

```
import java.util.ArrayList;

public class BadStack extends ArrayList<String> {

    public void push(String item) {
        add(item);
    }

    public String pop() {
        return remove(size() - 1);
    }

    public String peek() {
        return get(size() - 1);
    }
}
```

StackDemo.java

```
public class StackDemo {
    public static void main(String[] args) {

        // BAD version: the stack rules can be broken
        BadStack bad = new BadStack();
        bad.push("A");
        bad.push("B");
        bad.push("C");
```

```
        bad.add(0, "HACKED");        // inserts at the bottom, not allowed in a
real stack
        bad.remove(1);               // removes from the middle
        System.out.println("BadStack contents: " + bad);

        // GOOD version: only the four stack methods exist
        MyStack good = new MyStack();
        good.push("A");
        good.push("B");
        good.push("C");
        System.out.println("MyStack pop: " + good.pop());
        System.out.println("MyStack peek: " + good.peek());

        // good.add(0, "HACKED");    // COMPILE ERROR, MyStack has no add method
    }
}
```

Explanation

BadStack inherits add(index, item), remove(index), get(index), set and clear from ArrayList, so callers can insert or remove in the middle, which breaks the last-in-first-out rule. A stack is not a kind of ArrayList (is-a is false), it only uses a list (has-a). In MyStack the list is private, so only push, pop, peek and isEmpty are available.

Conclusion

Inheritance lets a subclass reuse and specialise a parent's code. The parent's constructor always runs first. Use inheritance only for a true is-a relationship, and prefer composition otherwise.

Questions

1) What happens if a superclass has no no-argument constructor and the subclass omits super(...)? The code does not compile, because Java inserts super() and finds no match.
2) Why does Java not allow multiple inheritance of classes? It avoids the diamond problem, where two parents give conflicting versions of the same method. Interfaces give multiple inheritance of type safely.
3) What does final mean on a variable, a method and a class? A final variable is assigned once, a final method cannot be overridden, and a final class cannot be extended.

4) When do you use composition instead of inheritance? When the relationship is has-a (a Car has an Engine), not is-a.

LAB 5- Polymorphism

Objectives

- Tell overloading (compile-time) apart from overriding (run-time).
- Use a superclass reference for subclass objects (upcasting) and explain dynamic dispatch.
- Downcast safely with instanceof.

Sample programs

Calculator java

```
public class Calculator {
// EXPLAIN: same name, different parameter lists -> resolved at compile time
int add(int a, int b) { return a + b; }
int add(int a, int b, int c) { return a + b + c; }
double add(double a, double b) { return a + b; }
String add(String a, String b) { return a + b; }
public static void main(String[] args) {
Calculator calc = new Calculator();
System.out.println(calc.add(2, 3)); // add(int,int) -> 5
System.out.println(calc.add(2, 3, 4)); // add(int,int,int) -> 9
System.out.println(calc.add(2.5, 3.5)); // add(double,double) -> 6.0
System.out.println(calc.add("Java", "OOP")); // add(String,String) -> JavaOOP

System.out.println(calc.add(2, 3.5)); // int widened to double -> 5.5

}
}
```

overloading, resolved by the compiler from the argument types. Output: 5, 9, 6.0, JavaOOP, 5.5.

Output:

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab5> javac Calculator.java

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab5> java Calculator

5

9

6.0

JavaOOP

5.5

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab5>

Shapes: an array of Shape holds a Circle, Rectangle and Triangle, and each area() runs the object's own version.

Shapes.java

```
class Shape {
String name() { return "Generic shape"; }
double area() { return 0; }
}
class Circle extends Shape {
private final double r;
Circle(double r) { this.r = r; }
@Override String name() { return "Circle"; }
@Override double area() { return Math.PI * r * r; }
}
class Rectangle extends Shape {
private final double w, h;
Rectangle(double w, double h) { this.w = w; this.h = h; }
@Override String name() { return "Rectangle"; }
@Override double area() { return w * h; }
}
class Triangle extends Shape {
private final double b, h;
Triangle(double b, double h) { this.b = b; this.h = h; }
@Override String name() { return "Triangle"; }
@Override double area() { return 0.5 * b * h; }
}
public class Shapes {
```

```
public static void main(String[] args) {
// EXPLAIN: upcasting -> one array type holds many different subclass objects
Shape[] shapes = {
new Circle(5),
new Rectangle(4, 6),
new Triangle(3, 8)
};
double total = 0;
for (Shape s : shapes) {
// EXPLAIN: JVM picks name()/area() from the ACTUAL object at run time
System.out.printf("%-10s area = %.2f%n", s.name(), s.area());
total += s.area();
}
System.out.printf("Total area = %.2f%n", total);
}
}
```

Output:

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab5> javac Shapes.java

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab5> java Shapes

Circle area = 78.54

Rectangle area = 24.00

Triangle area = 12.00

Total area = 114.54

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab5>

CastingDemo.java

```
class Animal {
String type = "Animal"; // field
void speak() { System.out.println("..."); }
static void info() { System.out.println("Animal info"); }
}
class Dog extends Animal {
String type = "Dog"; // hides the field, does not override it
```

```
@Override void speak() { System.out.println("Woof"); }
void fetch() { System.out.println("Fetching..."); }
static void info() { System.out.println("Dog info"); } // hides, not overrides
}
public class CastingDemo {
public static void main(String[] args) {
Animal a = new Dog(); // upcast (implicit, always safe)
a.speak(); // Woof -> method IS polymorphic
System.out.println(a.type); // Animal -> field follows the DECLARED type
a.info(); // Animal info -> static follows declared type
// a.fetch(); // COMPILE ERROR: Animal has no fetch()
// EXPLAIN: downcast needs a check, otherwise ClassCastException is possible
if (a instanceof Dog d) { // pattern matching (Java 16+): test AND cast
d.fetch();
}
Animal plain = new Animal();
if (!(plain instanceof Dog)) {
System.out.println("plain is not a Dog; downcast avoided");
}
}
}
```

Output:

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab5> javac CastingDemo.java

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab5> java CastingDemo

Woof

Animal

Animal info

Fetching...

plain is not a Dog; downcast avoided

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab5>

Exercises

Printer.java

```java
import java.util.Arrays;

public class Printer {

    void print(int x) {
        System.out.println("int: " + x);
    }

    void print(double x) {
        System.out.println("double: " + x);
    }

    void print(String x) {
        System.out.println("String: " + x);
    }

    void print(int[] x) {
        System.out.println("int[]: " + Arrays.toString(x));
    }

    public static void main(String[] args) {
        Printer p = new Printer();
        p.print(42);
        p.print(3.14);
        p.print("Hello");
        p.print(new int[]{1, 2, 3});
    }
}
```

Output:

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab5\exercises> javac Printer.java

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab5\exercises> java Printer

int: 42

double: 3.14

String: Hello

int[]: [1, 2, 3]

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab5\exercises>

Animal.java

```
public class Animal {
    String name;

    Animal(String name) {
        this.name = name;
    }

    String sound() {
        return "...";
    }
}
```

Cat.java

```
public class Cat extends Animal {

    Cat(String name) {
        super(name);
    }

    @Override
    String sound() {
        return "Meow";
    }
}
```

Cow.java

```
public class Cow extends Animal {

    Cow(String name) {
        super(name);
    }

    @Override
    String sound() {
```

```
        return "Moo";
    }
}
```

Duck.java

```
public class Duck extends Animal {

    Duck(String name) {
        super(name);
    }

    @Override
    String sound() {
        return "Quack";
    }
}
```

AnimalDemo.java

```
public class AnimalDemo {
    public static void main(String[] args) {
        // upcasting: one array type holds three different subclasses
        Animal[] animals = {
            new Cat("Tom"),
            new Cow("Bessie"),
            new Duck("Donald")
        };

        for (Animal a : animals) {
            System.out.println(a.name + " says " + a.sound());
        }
    }
}
```

Output:

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab5\exercises> javac *.java

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab5\exercises> java AnimalDemo

Tom says Meow

Bessie says Moo

Donald says Quack

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab5\exercises>

Explanation

The loop variable has type Animal, but Java runs the sound() of the actual object (dynamic method dispatch). Adding a Dog later needs no change to the loop.

Intern added to the employee hierarchy.

Intern.java

```
public class Intern extends Employee {

    private double stipend;

    public Intern(String name, double baseSalary, double stipend) {
        super(name, baseSalary);
        this.stipend = stipend;
    }

    @Override
    public double calculatePay() {
        return baseSalary + stipend;
    }
}
```

Developer.java

```
public class Developer extends Employee {
private int overtimeHours;
private static final double OVERTIME_RATE = 25.0;
public Developer(String name, double baseSalary, int overtimeHours) {
super(name, baseSalary);
this.overtimeHours = overtimeHours;
}
@Override
public double calculatePay() {
```

```
return baseSalary + overtimeHours * OVERTIME_RATE; // protected field used
directly

}
}
```

PayrollDemo.java

```
public class PayrollDemo {
    public static void main(String[] args) {
        Employee[] staff = {
            new Employee("Kudzai", 2000),
            new Manager("Nyasha", 3500, 800),
            new Developer("Tatenda", 3000, 10),
            new Intern("Rudo", 800, 200)
        };

        // the payroll loop: written once, never changed
        double total = 0;
        for (Employee e : staff) {
            e.describe();
            total += e.calculatePay();
        }
        System.out.printf("Total payroll: %.2f%n", total);
    }
}
```

Output:

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab5\exercises> cd ..

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab5> cd payroll

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab5\payroll> javac *.java

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab5\payroll> java PayrollDemo

Employee constructor: Kudzai

Employee constructor: Nyasha

Manager constructor: Nyasha

Employee constructor: Tatenda

Employee constructor: Rudo

Kudzai earns 2000.00

Nyasha earns 4300.00

Tatenda earns 3250.00

Rudo earns 1000.00

Total payroll: 10550.00

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab5\payroll>

Without the Intern line the total was 9550.00. The for loop was not changed.

Explanation

every subclass overrides calculatePay(), and Java picks the version from the actual object.

Method versus field through a superclass reference. Prediction: Child method, Parent field, Child field.

```
class Parent {
    String label = "Parent field";

    String who() {
        return "Parent method";
    }
}

class Child extends Parent {
    String label = "Child field";        // hides the parent's field, does not
override it

    @Override
    String who() {
        return "Child method";
```

```
    }
}

public class FieldVsMethod {
    public static void main(String[] args) {
        Parent p = new Child();          // Parent variable, Child object

        System.out.println(p.who());     // method call
        System.out.println(p.label);     // field access

        Child c = (Child) p;             // same object, seen through a Child
variable
        System.out.println(c.label);
    }
}
// The 3 lines of output are:
// Child method
// Parent field
// Child field
```

Output:

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab5> cd ex4

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab5\ex4> javac FieldVsMethod.java

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab5\ex4> java FieldVsMethod

Child method

Parent field

Child field

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab5\ex4>

Explanation

p.who() runs the Child version because methods are chosen from the actual object at run time. p.label gives Parent field because fields follow the declared type of the variable. After casting to Child, the same object shows Child field. Fields are hidden, not overridden.

PaymentProcessor, two designs.

Design 1 (instanceof pattern matching):

Payment.java

```
public class Payment {
    private double amount;

    public Payment(double amount) {
        this.amount = amount;
    }

    public double getAmount() { return amount; }
}
```

CardPayment.java

```
public class CardPayment extends Payment {
    private String last4;           // last 4 digits only, never the full number

    public CardPayment(String last4, double amount) {
        super(amount);
        this.last4 = last4;
    }

    public String getLast4() { return last4; }
}
```

MobileMoneyPayment.java

```
public class MobileMoneyPayment extends Payment {

    private String phoneNumber;

    public MobileMoneyPayment(String phoneNumber, double amount) {
        super(amount);
        this.phoneNumber = phoneNumber;
    }
```

```
    public String getPhoneNumber() { return phoneNumber; }
}
public class PaymentProcessor {

    // no instanceof and no if/else chain: each payment knows how to process
itself
    public void pay(Payment p) {
        p.process();
    }

    public static void main(String[] args) {
        PaymentProcessor processor = new PaymentProcessor();
        processor.pay(new CardPayment("4417", 50));
        processor.pay(new MobileMoneyPayment("0771234567", 20));
        processor.pay(new BankTransfer("ACC-001", 100));
    }
}
```

PaymentProcessor.java

```
public class PaymentProcessor {

    // no instanceof and no if/else chain: each payment knows how to process
itself
    public void pay(Payment p) {
        p.process();
    }

    public static void main(String[] args) {
        PaymentProcessor processor = new PaymentProcessor();
        processor.pay(new CardPayment("4417", 50));
        processor.pay(new MobileMoneyPayment("0771234567", 20));
        processor.pay(new BankTransfer("ACC-001", 100));
    }
}
```

Output:

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab5\ex5a> javac *.java

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab5\ex5a> java PaymentProcessor

Charging card ending 4417: $50.0

Sending mobile money request to 0771234567: $20.0

Unknown payment type

Design 2 (abstract Payment with process()):

Payment.java

```
public abstract class Payment {
    private double amount;

    public Payment(double amount) {
        this.amount = amount;
    }

    public double getAmount() { return amount; }

    // the contract: every payment type MUST say how it is processed
    public abstract void process();
}
```

MobileMoneyPayment.java

```
public class MobileMoneyPayment extends Payment {
    private String phoneNumber;

    public MobileMoneyPayment(String phoneNumber, double amount) {
        super(amount);
        this.phoneNumber = phoneNumber;
    }

    public String getPhoneNumber() { return phoneNumber; }

    @Override
    public void process() {
        System.out.println("Sending mobile money request to " + phoneNumber + ":
$" + getAmount());
    }
}
```

CardPayment.java

```
public class CardPayment extends Payment {
    private String last4;           // last 4 digits only, never the full number

    public CardPayment(String last4, double amount) {
        super(amount);
        this.last4 = last4;
    }

    public String getLast4() { return last4; }

    @Override
    public void process() {
        System.out.println("Charging card ending " + last4 + ": $" +
getAmount());
    }
}
```

BankTransfer.java

```
public class BankTransfer extends Payment {
    private String accountNumber;

    public BankTransfer(String accountNumber, double amount) {
        super(amount);
        this.accountNumber = accountNumber;
    }

    public String getAccountNumber() { return accountNumber; }

    @Override
    public void process() {
        System.out.println("Bank transfer to account " + accountNumber + ": $" +
getAmount());
    }
}
```

PaymentProcessor.java

```
public class PaymentProcessor {

    // no instanceof and no if/else chain: each payment knows how to process
itself
    public void pay(Payment p) {
```

```
        p.process();
    }

    public static void main(String[] args) {
        PaymentProcessor processor = new PaymentProcessor();
        processor.pay(new CardPayment("4417", 50));
        processor.pay(new MobileMoneyPayment("0771234567", 20));
        processor.pay(new BankTransfer("ACC-001", 100));
    }
}
```

Output:

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab5> cd ex5b

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab5\ex5b> javac *.java

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab5\ex5b> java PaymentProcessor

Charging card ending 4417: $50.0

Sending mobile money request to 0771234567: $20.0

Bank transfer to account ACC-001: $100.0

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab5\ex5b>

Comparison

in Design 1 all the logic is in one pay() method, which is easy to read at first, but every new payment type means editing pay(), and a forgotten case falls into Unknown. In Design 2 each class owns its behaviour, so adding BankTransfer needed no change to pay(), and the compiler forces every payment to implement process(). Design 2 follows open for extension, closed for modification. instanceof can still be reasonable for classes you cannot change.

Conclusion

Polymorphism lets one call behave differently depending on the real object. It makes code easy to extend, because new subclasses work with existing loops and methods.

Viva answers

1) Difference between overloading and overriding? Overloading is the same method name with different parameters in one class, chosen at compile time. Overriding is a subclass redefining an inherited method with the same signature, chosen at run time.
2) Explain dynamic method dispatch. Java chooses which version of an overridden method to run from the actual object at run time, not from the variable's declared type.
3) Why are fields and static methods not polymorphic? They are resolved from the declared type at compile time. Fields are hidden, not overridden.
4) Why use instanceof before a downcast? To avoid a ClassCastException.

Lab 6: Abstraction, Abstract Classes and Interfaces

Objectives

1) Define abstract classes and methods, and explain why they cannot be instantiated.
2) Define and implement interfaces, including default and static methods.
3) Choose between an abstract class and an interface, and program to an interface.

Sample programs

VehicleDemo.java

VehicleDemo (abstract class): TRK-1: 100 km costs 80.00 and MB-7: 100 km costs 15.00.

```
abstract class Vehicle {
private final String plate;
Vehicle(String plate) { this.plate = plate; } // EXPLAIN: abstract classes CAN
have constructors

// EXPLAIN: abstract -> no body; every concrete subclass MUST provide one
```

```
abstract double fuelCostPerKm();
// concrete method shared by all vehicles (template of common behaviour)
double tripCost(double km) {
return km * fuelCostPerKm();
}
String getPlate() { return plate; }
}
class Truck extends Vehicle {
Truck(String plate) { super(plate); }
@Override double fuelCostPerKm() { return 0.80; }
}
class Motorbike extends Vehicle {
Motorbike(String plate) { super(plate); }
@Override double fuelCostPerKm() { return 0.15; }
}
public class VehicleDemo {
public static void main(String[] args) {
// Vehicle v = new Vehicle("X"); // COMPILE ERROR: Vehicle is abstract
Vehicle[] fleet = { new Truck("TRK-1"), new Motorbike("MB-7") };
for (Vehicle v : fleet) {
System.out.printf("%s: 100 km costs %.2f%n", v.getPlate(),
v.tripCost(100));
}
}
}
```

Output:

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab6> javac *.java

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab6> java VehicleDemo

TRK-1: 100 km costs 80.00

MB-7: 100 km costs 15.00

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab6>

InterfaceDemo.java

```
interface Payable {
double MAX_LIMIT = 10_000; // implicitly public static final
void pay(double amount); // implicitly public abstract
```

```
// EXPLAIN: default method gives implementations for free; classes may override
it

default void receipt(double amount) {
System.out.println("Receipt: paid " + amount);
}
// EXPLAIN: static utility method belongs to the interface itself
static boolean withinLimit(double amount) {
return amount <= MAX_LIMIT;
}
}
interface Loggable {
void log(String message);
}
// EXPLAIN: one class, several interfaces -> multiple inheritance of TYPE
class CardPayment implements Payable, Loggable {
@Override
public void pay(double amount) {
if (!Payable.withinLimit(amount)) {
log("Blocked: over limit");
return;
}
log("Card charged " + amount);
receipt(amount);
}
@Override
public void log(String message) { // interface methods must be public
System.out.println("[LOG] " + message);
}
}
class CashPayment implements Payable {
@Override
public void pay(double amount) {
System.out.println("Cash received: " + amount);
}
@Override
public void receipt(double amount) { // overriding the default
System.out.println("Handwritten receipt for " + amount);
}
}

public class InterfaceDemo {
public static void main(String[] args) {
// EXPLAIN: program to the interface: the variable type is Payable
Payable[] methods = { new CardPayment(), new CashPayment() };
```

```
for (Payable p : methods) {
p.pay(250);
p.receipt(250);
System.out.println("---");
}
new CardPayment().pay(50_000);
}
}
```

Output:

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab6> java InterfaceDemo

[LOG] Card charged 250.0

Receipt: paid 250.0

Receipt: paid 250.0

Cash received: 250.0

Handwritten receipt for 250.0

[LOG] Blocked: over limit

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab6>

ComparableDemo.java

```
import java.util.Arrays;
class Player implements Comparable<Player> {
private final String name;
private final int score;
Player(String name, int score) { this.name = name; this.score = score; }
// EXPLAIN: defines the natural ordering: highest score first
@Override
public int compareTo(Player other) {
```

```
return Integer.compare(other.score, this.score);
}
@Override
public String toString() { return name + "(" + score + ")"; }
}
public class ComparableDemo {
public static void main(String[] args) {
Player[] players = {
new Player("Anesu", 70), new Player("Blessing", 95), new Player("Chenai",
82)
};
Arrays.sort(players); // sort relies only on the Comparable contract
System.out.println(Arrays.toString(players));
}
}
```

Output:

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab6> java ComparableDemo

[Blessing(95), Chenai(82), Anesu(70)]

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab6>

Exercises

Shape.java

```
public abstract class Shape {

    // no body: every concrete subclass MUST provide these
    public abstract double area();
    public abstract double perimeter();
}
```

Rectangle.java

```
public class Rectangle extends Shape {

    private double width;
    private double height;
```

```
    public Rectangle(double width, double height) {
        this.width = width;
        this.height = height;
    }

    @Override
    public double area() {
        return width * height;
    }

    @Override
    public double perimeter() {
        return 2 * (width + height);
    }
}
```

Square.java

```
public class Square extends Shape {

    private double side;

    public Square(double side) {
        this.side = side;
    }

    @Override
    public double area() {
        return side * side;
    }

    @Override
    public double perimeter() {
        return 4 * side;
    }
}
```

Circle.java

```
public class Circle extends Shape {

    private double radius;

    public Circle(double radius) {
```

```
        this.radius = radius;
    }

    @Override
    public double area() {
        return Math.PI * radius * radius;
    }

    @Override
    public double perimeter() {
        return 2 * Math.PI * radius;
    }
}
```

ShapeDemo.java

```
public class ShapeDemo {
    public static void main(String[] args) {
        Shape[] shapes = {
            new Circle(5),
            new Rectangle(4, 6),
            new Square(3)
        };

        for (Shape s : shapes) {
            System.out.printf("%-10s area = %.2f, perimeter = %.2f%n",
                    s.getClass().getSimpleName(), s.area(), s.perimeter());
        }

         //Shape s = new Shape();    // Error: Shape is abstract, cannot be
instantiated
    }
}
```

Output:

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab6\exercises> javac *.java

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab6\exercises> java ShapeDemo

Circle area = 78.54, perimeter = 31.42

Rectangle area = 24.00, perimeter = 20.00

Square area = 9.00, perimeter = 12.00

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab6\exercises>

Flyable interface for Bird and Aeroplane.

Flyable.java

```
public interface Flyable {
    void fly();          // implicitly public abstract
}
```

Bird.java

```
public class Bird implements Flyable {

    @Override
    public void fly() {
        System.out.println("The bird flaps its wings and flies");
    }
}
```

Aeroplane.java

```
public class Aeroplane implements Flyable {

    @Override
    public void fly() {
        System.out.println("The aeroplane takes off from the runway");
    }
}
```

FlyDemo.java

```
public class FlyDemo {
    public static void main(String[] args) {
        // the variable type is the interface, not the class
        Flyable[] things = { new Bird(), new Aeroplane() };

        for (Flyable f : things) {
```

```
            f.fly();
        }
    }
}
```

Output:

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab6\exercises> java FlyDemo

The bird flaps its wings and flies

The aeroplane takes off from the runway

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab6\exercises>

Explanation

Bird and Aeroplane share no superclass, and a class can extend only one class but implement many interfaces, so an interface is the right tool.

Abstract Account with concrete deposit() and abstract monthlyFee(). SavingsAccount charges 0 at a balance of 1000 or more, otherwise 5. CurrentAccount charges a flat 15.

Account.java

```
public abstract class Account {

    private final String accountNumber;
    private double balance;

    public Account(String accountNumber, double openingBalance) {
        this.accountNumber = accountNumber;
        this.balance = openingBalance;
    }

    public String getAccountNumber() { return accountNumber; }
    public double getBalance()       { return balance; }

    // concrete: written once, shared by every account type
```

```
    public void deposit(double amount) {
        if (amount <= 0) {
            throw new IllegalArgumentException("Deposit must be positive");
        }
        balance += amount;
    }

    // abstract: no body, every subclass MUST decide its own fee
    public abstract double monthlyFee();

    // concrete, but it relies on the abstract method
    public void applyMonthlyFee() {
        balance -= monthlyFee();
    }
}
```

SavingsAccount.java

```
public class SavingsAccount extends Account {

    public SavingsAccount(String accountNumber, double openingBalance) {
        super(accountNumber, openingBalance);
    }

    // free if the balance is 1000 or more, otherwise 5.00
    @Override
    public double monthlyFee() {
        if (getBalance() >= 1000) {
            return 0.0;
        }
        return 5.0;
    }
}
```

CurrentAccount.java

```
public class CurrentAccount extends Account {

    public CurrentAccount(String accountNumber, double openingBalance) {
        super(accountNumber, openingBalance);
    }

    // flat fee: always 15.00, whatever the balance
```

```
    @Override
    public double monthlyFee() {
        return 15.0;
    }
}
```

AccountDemo.java

```
public class AccountDemo {
    public static void main(String[] args) {
        Account[] accounts = {
            new SavingsAccount("SAV-001", 1200),
            new SavingsAccount("SAV-002", 300),
            new CurrentAccount("CUR-001", 500)
        };

        for (Account a : accounts) {
            a.deposit(100);
            double fee = a.monthlyFee();
            a.applyMonthlyFee();
            System.out.printf("%s (%s): fee %.2f, balance now %.2f%n",
                    a.getAccountNumber(), a.getClass().getSimpleName(), fee,
a.getBalance());
        }

        try {
            accounts[0].deposit(-5);
        } catch (IllegalArgumentException e) {
            System.out.println("Rejected: " + e.getMessage());
        }

        //Account a = new Account("X", 0);    // Error
    }
}
```

Output:

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab6\exercises> java AccountDemo

SAV-001 (SavingsAccount): fee 0.00, balance now 1300.00

SAV-002 (SavingsAccount): fee 5.00, balance now 395.00

CUR-001 (CurrentAccount): fee 15.00, balance now 585.00

Rejected: Deposit must be positive

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab6\exercises>

Explanation

Account is abstract because it holds the code every account shares (deposit and the balance), but a general Account has no fee, so monthlyFee() is left abstract and each subclass must provide it.

Student implements Comparable (by name) plus a Comparator (by marks).

StudentSortDemo.java

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.Comparator;
import java.util.List;

public class StudentSortDemo {
    public static void main(String[] args) {
        List<Student> list = new ArrayList<>(List.of(
            new Student("Kuda", 65),
            new Student("Ada", 90),
            new Student("Lynn", 72),
            new Student("Blessing", 58)
        ));

        // 1) natural order, from Comparable (by name)
        Collections.sort(list);
        System.out.println("By name: " + list);

        // 2) Comparator: an outside ordering by marks, highest first
        Comparator<Student> byMarks =
Comparator.comparingInt(Student::getMarks).reversed();
        list.sort(byMarks);
        System.out.println("By marks (high to low): " + list);

        // 3) the same Comparator idea, lowest first
        list.sort(Comparator.comparingInt(Student::getMarks));
```

```
        System.out.println("By marks (low to high): " + list);
    }
}
```

Output:

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab6\exercises> java StudentSortDemo

By name: [Ada(90), Blessing(58), Kuda(65), Lynn(72)]

By marks (high to low): [Ada(90), Lynn(72), Kuda(65), Blessing(58)]

By marks (low to high): [Blessing(58), Kuda(65), Lynn(72), Ada(90)]

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab6\exercises>

Explanation

Comparable gives the class one natural order, written inside the class. A Comparator is a separate object that supplies another order without changing the class, and a class can have many of them.

Two default methods with the same name. The class overrides hello() and chooses with EnglishGreeter.super.hello() and ShonaGreeter.super.hello().

InterfaceConflict.java

```
interface EnglishGreeter {
    default void hello() {
        System.out.println("Hello!");
    }
}

interface ShonaGreeter {
    default void hello() {
        System.out.println("Mhoroi!");
    }
}
```

Output:

```
PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab6\ex5> javac InterfaceConflict.java

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab6\ex5> java InterfaceConflict

Hello!

Mhoroi!

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab6\ex5>
```

Error without the override: class Greeter inherits unrelated defaults for hello() from types EnglishGreeter and ShonaGreeter.

Explanation

If a class inherits two default methods with the same signature, the compiler cannot choose, so the class must override the method. Inside it, InterfaceName.super.method() picks a specific version. If the class itself defines the method, the class version wins.

Conclusion

Abstraction shows what an object does and hides how. Abstract classes share code and state among related classes, and interfaces describe a capability that unrelated classes can share.

Viva answers

1) When would you choose an abstract class over an interface? When closely related classes need shared code or state. Use an interface for a capability that unrelated classes share.
2) Why can an abstract class not be instantiated? It is deliberately incomplete, because some methods have no body.
3) Why must interface methods be public when implemented? Interface methods are public, and an implementation cannot reduce visibility.
4) What is a default method? A method with a body inside an interface, which implementing classes get for free and may override.

Lab 7: Exception Handling

Objectives

1) Explain checked and unchecked exceptions.
2) Use try, catch, finally, throw and throws.
3) Create custom exceptions and use try-with-resources.

Sample programs

ExceptionsBasic.java

```
public class ExceptionBasics {
static int divide(int a, int b) {
return a / b; // ArithmeticException if b == 0
}
public static void main(String[] args) {
int[] data = {10, 20, 30};
String text = null;
try {
System.out.println("10 / 2 = " + divide(10, 2));
System.out.println("10 / 0 = " + divide(10, 0)); // throws here
System.out.println("never printed");
} catch (ArithmeticException e) {
System.out.println("Math problem: " + e.getMessage());
} finally {
System.out.println("finally always runs");
}
// EXPLAIN: several catch blocks, specific first
try {
System.out.println(data[5]);
System.out.println(text.length());
} catch (ArrayIndexOutOfBoundsException e) {
System.out.println("Bad index: " + e.getMessage());
} catch (NullPointerException e) {
System.out.println("Null reference");
} catch (Exception e) { // most general last
System.out.println("Something else: " + e);
}
// EXPLAIN: multi-catch when the handling is identical
try {
Integer.parseInt("abc");
} catch (NumberFormatException | NullPointerException e) {
System.out.println("Invalid number input");
```

```
}
}
}
```

Output:

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab7> javac ExceptionBasics.java

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab7> java ExceptionBasics

10 / 2 = 5

Math problem: / by zero

finally always runs

Bad index: Index 5 out of bounds for length 3

Invalid number input

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab7>

InsufficientFundsexception

```
// EXPLAIN: extending Exception makes this a CHECKED exception
public class InsufficientFundsException extends Exception {
private final double shortfall;
public InsufficientFundsException(String message, double shortfall) {
super(message); // pass the message to Exception
this.shortfall = shortfall;
}
public double getShortfall() { return shortfall; }
}
```

Wallet.java

```
public class Wallet {
```

```java
private double balance;
public Wallet(double balance) { this.balance = balance; }
// EXPLAIN: 'throws' warns callers they must handle this checked exception
public void spend(double amount) throws InsufficientFundsException {
if (amount > balance) {
// EXPLAIN: 'throw' creates and throws the exception object
throw new InsufficientFundsException(
"Cannot spend " + amount + "; balance is " + balance, amount -
balance);
}
balance -= amount;
}
public double getBalance() { return balance; }
public static void main(String[] args) {
Wallet w = new Wallet(100);
try {
w.spend(60);
System.out.println("Balance now: " + w.getBalance());
w.spend(60); // fails
} catch (InsufficientFundsException e) {
System.out.println(e.getMessage());
System.out.println("Short by: " + e.getShortfall());
}
}
}
```

Output:

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab7> javac *.java

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab7> java Wallet

Balance now: 40.0

Cannot spend 60.0; balance is 40.0

Short by: 20.0

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab7>

ResourceDemo.java

```
class Connection implements AutoCloseable { // EXPLAIN: AutoCloseable enables
try-with-resources

private final String name;
Connection(String name) {
this.name = name;
System.out.println(name + " opened");
}
void query(boolean fail) {
if (fail) throw new IllegalStateException(name + " query failed");
System.out.println(name + " query ok");
}
@Override
public void close() {
System.out.println(name + " closed"); // called automatically
}
}
public class ResourceDemo {
public static void main(String[] args) {
// EXPLAIN: resources close in REVERSE order, even if an exception occurs
try (Connection c1 = new Connection("DB1");
Connection c2 = new Connection("DB2")) {
c1.query(false);
c2.query(true);
} catch (IllegalStateException e) {
System.out.println("Caught: " + e.getMessage());
}
// EXPLAIN: wrapping a low-level exception keeps the original as the cause
try {
try {
Integer.parseInt("12x");
} catch (NumberFormatException e) {
throw new RuntimeException("Configuration invalid", e);
}
} catch (RuntimeException e) {
System.out.println(e.getMessage() + " <- " +
e.getCause().getClass().getSimpleName());
}
}
}
```

Output

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab7> java ResourceDemo

DB1 opened

DB2 opened

DB1 query ok

DB2 closed

DB1 closed

Caught: DB2 query failed

Configuration invalid <- NumberFormatException

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab7>

Exercises

DivideDemo.java

```java
import java.util.Scanner;

public class DivideDemo {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);

        try {
            System.out.print("Enter first number: ");
            int a = Integer.parseInt(in.nextLine());

            System.out.print("Enter second number: ");
            int b = Integer.parseInt(in.nextLine());

            System.out.println(a + " / " + b + " = " + (a / b));

        } catch (NumberFormatException e) {
            System.out.println("That is not a whole number");
        } catch (ArithmeticException e) {
            System.out.println("Cannot divide by zero");
        } finally {
            in.close();
            System.out.println("Done");
```

```
        }
    }
}
```

Output:

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab7> java DivideDemo

Enter first number: 10

Enter second number: 0

Cannot divide by zero

Done

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab7>

Array access that catches ArrayIndexOutOfBoundsException.

ArrayAccess.java

```
public class ArrayAccess {

    // returns the element, or -1 if the index is invalid
    static int getElement(int[] data, int index) {
        try {
            return data[index];
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Bad index " + index + ": " + e.getMessage());
            return -1;
        }
    }

    public static void main(String[] args) {
        int[] scores = {70, 85, 90};

        System.out.println(getElement(scores, 1));
        System.out.println(getElement(scores, 5));
        System.out.println(getElement(scores, -1));
    }
}
```

Output:

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab7> java ArrayAccess

85

Bad index 5: Index 5 out of bounds for length 3

-1

Bad index -1: Index -1 out of bounds for length 3

-1

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab7>

InvalidAgeException (checked) and validateAge.

```java
public class InvalidAgeException extends Exception {

    private final int age;

    public InvalidAgeException(String message, int age) {
        super(message);          // pass the message up to Exception
        this.age = age;
    }

    public int getAge() { return age; }
}
```

AgeCheck.java

```java
public class AgeCheck {

    // 'throws' warns callers that this method may throw a checked exception
    static void validateAge(int age) throws InvalidAgeException {
        if (age < 0 || age > 120) {
            throw new InvalidAgeException("Age must be between 0 and 120", age);
        }
        System.out.println("Age " + age + " is valid");
    }

    public static void main(String[] args) {
        int[] tests = {25, 0, 120, -5, 121};
```

```
        for (int t : tests) {
            try {
                validateAge(t);
            } catch (InvalidAgeException e) {
                System.out.println("Invalid age " + e.getAge() + ": " +
e.getMessage());
            }
        }
    }
}
```

Output:

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab7> java AgeCheck

Age 25 is valid

Age 0 is valid

Age 120 is valid

Invalid age -5: Age must be between 0 and 120

Invalid age 121: Age must be between 0 and 120

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab7>

BankAccount.withdraw throws InsufficientFundsException.

```
public class BankExceptionDemo {
    public static void main(String[] args) {
        BankAccount acc = new BankAccount("ACC-001", "Tendai", 500.00);

        try {
            acc.withdraw(100);
            System.out.println("Withdrew 100, balance: " + acc.getBalance());
            acc.withdraw(5000);
            System.out.println("This line is never reached");
        } catch (InsufficientFundsException e) {
            System.out.println(e.getMessage());
            System.out.println("Short by: " + e.getShortfall());
```

```
        }

        try {
            acc.withdraw(-20);
        } catch (InsufficientFundsException e) {
            System.out.println("Funds problem: " + e.getMessage());
        } catch (IllegalArgumentException e) {
            System.out.println("Rejected: " + e.getMessage());
        }
    }
}
```

Output:

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab7\ex4> javac *.java

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab7\ex4> java BankExceptionDemo.java

Withdrew 100, balance: 400.0

Cannot withdraw 5000.0; balance is 400.0

Short by: 4600.0

Rejected: Withdrawal must be positive

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab7\ex4>

Trade-offs

Returning false is simple and needs no try/catch, but it is easy to ignore, so a failure can go unnoticed, and it gives no detail. Throwing a checked exception cannot be ignored, carries details such as the shortfall, and separates the normal path from the error path. But every caller must handle it, and old callers must be updated. For money, where a silent failure is dangerous, an exception is the safer choice.

Return inside try with a finally

```
public class FinallyDemo {
```

```java
// Test 1: return inside try, print inside finally
static int test1() {
    try {
        System.out.println("in try");
        return 1;
    } finally {
        System.out.println("in finally");
    }
}

// Test 2: finally changes the variable AFTER return has picked its value
static int test2() {
    int x = 10;
    try {
        return x;
    } finally {
        x = 99;
        System.out.println("finally set x to " + x);
    }
}

// Test 3: an exception, a catch with its own return, and a finally
static int test3() {
    try {
        System.out.println("in try, about to divide");
        int r = 10 / 0;
        return r;
    } catch (ArithmeticException e) {
        System.out.println("Caught: " + e.getMessage());
        return -1;
    } finally {
        System.out.println("in finally");
    }
}

public static void main(String[] args) {
    System.out.println("--- test 1 ---");
    System.out.println("returned " + test1());

    System.out.println("--- test 2 ---");
    System.out.println("returned " + test2());

    System.out.println("--- test 3 ---");
    System.out.println("returned " + test3());
```

```
    }
}

//the lines that will print are:
// --- test 1 ---
// in try
// in finally
// returned 1
// --- test 2 ---
// in try
// finally set x to 99
// returned 10
// --- test 3 ---
// in try, about to divide
// Caught: / by zero
// in finally
// returned -1
```

Output:

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab7> cd ex5

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab7\ex5> javac FinallyDemo.java

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab7\ex5> java FinallyDemo

--- test 1 ---

in try

in finally

returned 1

--- test 2 ---

finally set x to 99

returned 10

--- test 3 ---

in try, about to divide

Caught: / by zero

in finally

returned -1

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab7\ex5>

Explanation

Finally always runs, whether the try finishes normally, hits return, or throws. With return, Java works out the return value first, then runs finally, then returns. In test 2 the value 10 was already set aside, so changing x to 99 does not change what is returned.

Conclusion

Exceptions let a program respond to errors instead of crashing. Checked exceptions force the caller to handle recoverable problems, and finally and try-with-resources guarantee cleanup.

Viva answers

1) Checked versus unchecked exceptions? Checked ones (IOException, SQLException, custom ones that extend Exception) must be caught or declared. Unchecked ones (NullPointerException, ArithmeticException) extend RuntimeException and are optional to catch.
2) throw versus throws? throw creates and throws one exception object. throws in a method signature declares which checked exceptions the method may throw.
3) How does try-with-resources differ from try/finally? Resources that implement AutoCloseable are closed automatically, in reverse order, even if an exception occurs.
4) Why order catch blocks from specific to general? A general catch placed first makes the later ones unreachable, and it does not compile.

Lab 8: Generics and the Collections Framework

Objectives

1) Write generic classes and methods, including bounded type parameters.
2) Choose the right collection: List, Set, Map, Queue.
3) Sort with Comparable and Comparator.

Sample programs

GenericsDemo.java

```
import java.util.List;
// EXPLAIN: T is a type parameter chosen by the caller
class Box<T> {
private T content;
Box(T content) { this.content = content; }
T get() { return content; }
void set(T value) { this.content = value; }
}
// EXPLAIN: two type parameters
class Pair<K, V> {
private final K key;
private final V value;
Pair(K key, V value) { this.key = key; this.value = value; }
K getKey() { return key; }
V getValue() { return value; }
@Override public String toString() { return key + "=" + value; }
}
public class GenericsDemo {
// EXPLAIN: generic METHOD; <T> before the return type declares the parameter
static <T> void printAll(List<T> items) {
for (T item : items) System.out.print(item + " ");
System.out.println();
}
// EXPLAIN: bounded type: T must be a Comparable<T>, so compareTo is available
static <T extends Comparable<T>> T maxOf(List<T> items) {
T best = items.get(0);
for (T item : items) {
if (item.compareTo(best) > 0) best = item;
}
return best;
}
public static void main(String[] args) {
Box<String> sb = new Box<>("Hello");
Box<Integer> ib = new Box<>(42); // autoboxing: int -> Integer
String s = sb.get(); // no cast needed
int n = ib.get() + 1;
```

```
// sb.set(99); // COMPILE ERROR: type safety
System.out.println(s + " " + n);
Pair<String, Integer> p = new Pair<>("age", 21);
System.out.println(p);
printAll(List.of(1, 2, 3));
printAll(List.of("a", "b"));
System.out.println(maxOf(List.of(3, 9, 4)));
System.out.println(maxOf(List.of("pear", "apple", "zebra")));
}
}
```

Output:

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab8> javac *.java

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab8> java GenericsDemo

Hello 43

age=21

1 2 3

a b

9

zebra

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab8>

CollectionsDemo.java

```
import java.util.*;
public class CollectionsDemo {
public static void main(String[] args) {
// ---- List: ordered, allows duplicates ----
List<String> names = new ArrayList<>(); // programming to the interface
names.add("Zodwa");
names.add("Alice");
names.add("Moyo");
names.add("Alice");
System.out.println("List: " + names + " size=" + names.size());
names.remove("Moyo");
```

```
Collections.sort(names); // natural (alphabetical) order
System.out.println("Sorted: " + names);
// ---- Set: no duplicates ----
Set<String> unique = new TreeSet<>(names); // TreeSet keeps elements sorted
System.out.println("Set: " + unique);
// ---- Map: key -> value ----
Map<String, Integer> marks = new HashMap<>();
marks.put("Tino", 78);
marks.put("Rufaro", 91);
marks.put("Tino", 82); // same key -> value replaced
marks.putIfAbsent("Nomsa", 65);
System.out.println("Tino: " + marks.get("Tino"));
System.out.println("Has Nomsa? " + marks.containsKey("Nomsa"));
// iterate a map in key order using TreeMap
for (Map.Entry<String, Integer> e : new TreeMap<>(marks).entrySet()) {
System.out.println(e.getKey() + " -> " + e.getValue());
}
// ---- Word frequency: a classic Map use ----
String text = "to be or not to be";
Map<String, Integer> freq = new TreeMap<>();
for (String w : text.split(" ")) {
freq.merge(w, 1, Integer::sum); // add 1, or start at 1
}
System.out.println(freq);
// ---- Queue / Deque ----
Deque<Integer> stack = new ArrayDeque<>();
stack.push(1); stack.push(2); stack.push(3);
System.out.println("Stack pop: " + stack.pop()); // LIFO -> 3
Queue<String> line = new ArrayDeque<>();
line.offer("first"); line.offer("second");
System.out.println("Queue poll: " + line.poll()); // FIFO -> first
}
}
```

Output:

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab8> java CollectionsDemo

List: [Zodwa, Alice, Moyo, Alice] size=4

Sorted: [Alice, Alice, Zodwa]

Set: [Alice, Zodwa]

Tino: 82

Has Nomsa? true

Nomsa -> 65

Rufaro -> 91

Tino -> 82

{be=2, not=1, or=1, to=2}

Stack pop: 3

Queue poll: first

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab8>

a List allows duplicates, a Set does not, a Map links keys to values, a stack is last-in-first-out and a queue is first-in-first-out.

SortingDemo.java

```
import java.util.*;
class Student implements Comparable<Student> {
private final String name;
private final double gpa;
Student(String name, double gpa) { this.name = name; this.gpa = gpa; }
String getName() { return name; }
double getGpa() { return gpa; }
// EXPLAIN: natural order = by name (one default ordering per class)
@Override
public int compareTo(Student o) { return name.compareTo(o.name); }
@Override public String toString() { return name + "(" + gpa + ")"; }
}
public class SortingDemo {
public static void main(String[] args) {
List<Student> list = new ArrayList<>(List.of(
new Student("Kuda", 3.2), new Student("Ada", 3.9), new Student("Lynn",
3.5)));
Collections.sort(list); // uses compareTo
```

```
System.out.println("By name: " + list);
// EXPLAIN: Comparator = an external, alternative ordering (any number of them)

list.sort(Comparator.comparingDouble(Student::getGpa).reversed());
System.out.println("By GPA desc: " + list);
list.sort(Comparator.comparing(Student::getName).reversed());
System.out.println("By name desc: " + list);
}
}
```

Output:

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab8> javac *.java

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab8> java SortingDemo

By name: [Ada(3.9), Kuda(3.2), Lynn(3.5)]

By GPA desc: [Ada(3.9), Lynn(3.5), Kuda(3.2)]

By name desc: [Lynn(3.5), Kuda(3.2), Ada(3.9)]

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab8>

Exercises

ListStats.java

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class ListStats {
    public static void main(String[] args) {
        List<Integer> nums = new ArrayList<>();
        int[] values = {12, 7, 25, 3, 18, 9, 30, 4, 21, 15};
        for (int v : values) {
            nums.add(v);
        }

        int sum = 0;
        for (int n : nums) {
```

```
            sum += n;
        }

        int max = nums.get(0);
        for (int n : nums) {
            if (n > max) {
                max = n;
            }
        }

        // copy first, so the original list keeps its order
        List<Integer> reversed = new ArrayList<>(nums);
        Collections.reverse(reversed);

        System.out.println("List: " + nums);
        System.out.println("Sum: " + sum);
        System.out.println("Max: " + max);
        System.out.println("Reversed: " + reversed);
    }
}
```

Output:

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab8\exercises> java ListStats

List: [12, 7, 25, 3, 18, 9, 30, 4, 21, 15]

Sum: 144

Max: 30

Reversed: [15, 21, 4, 30, 9, 18, 3, 25, 7, 12]

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab8\exercises>

Remove duplicates with a set

```
import java.util.ArrayList;
import java.util.List;
import java.util.Set;
import java.util.LinkedHashSet;

public class UniqueNames {
```

```
    public static void main(String[] args) {
        List<String> names = new ArrayList<>(
            List.of("Tariro", "Farai", "Tariro", "Rudo", "Farai", "Kuda"));

        Set<String> unique = new LinkedHashSet<>(names);

        System.out.println("Original: " + names + " (" + names.size() + ")");
        System.out.println("Unique: " + unique + " (" + unique.size() + ")");
    }
}
```

Output:

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab8\exercises> java UniqueNames

Original: [Tariro, Farai, Tariro, Rudo, Farai, Kuda] (6)

Unique: [Tariro, Farai, Rudo, Kuda] (4)

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab8\exercises>

StackDemo.java

```
import java.util.ArrayList;

public class Stack<T> {

    // HAS-A: the list is hidden inside
    private final ArrayList<T> items = new ArrayList<>();

    public void push(T item) {
        items.add(item);
    }

    public T pop() {
        if (items.isEmpty()) {
            throw new IllegalStateException("Stack is empty");
        }
        return items.remove(items.size() - 1);
    }
```

```
    public T peek() {
        if (items.isEmpty()) {
            throw new IllegalStateException("Stack is empty");
        }
        return items.get(items.size() - 1);
    }

    public boolean isEmpty() {
       return items.isEmpty();
    }
}
```

Output:

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab8\exercises> java StackDemo

pop: C

peek: B

empty? false

popped int: 20

empty? true

Caught: Stack is empty

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab8\exercises>

Common Error:

incompatible types: int cannot be converted to String. Explanation: Stack<T> is written once and works for any type, and the compiler catches wrong types at compile time and removes the need for casts. Because it uses an ArrayList inside, only push, pop, peek and isEmpty are exposed.

Word frequency, sorted by frequency descending. For the sentence: The cat sat on the mat. The dog sat on the log. A cat and a dog met.

```java
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

public class WordFrequency {
    public static void main(String[] args) {
        String text = "The cat sat on the mat. The dog sat on the log. A cat and
a dog met.";

        // Step 1: count each word
        Map<String, Integer> freq = new HashMap<>();
        for (String w : text.toLowerCase().split("[^a-z]+")) {
            if (!w.isEmpty()) {
                freq.merge(w, 1, Integer::sum);      // add 1, or start at 1
            }
        }

        // Step 2: copy the entries into a list and sort it
        List<Map.Entry<String, Integer>> entries = new
ArrayList<>(freq.entrySet());

        entries.sort((a, b) -> {
            int byCount = b.getValue().compareTo(a.getValue());    // highest
count first
            if (byCount != 0) {
                return byCount;
            }
            return a.getKey().compareTo(b.getKey());      // ties: alphabetical
        });

        for (Map.Entry<String, Integer> e : entries) {
            System.out.println(e.getKey() + ": " + e.getValue());
        }
    }
```

Output:

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab8\exercises> java WordFrequency

the: 4

a: 2

cat: 2

dog: 2

on: 2

sat: 2

and: 1

log: 1

mat: 1

met: 1

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab8\exercises>

Explanation

The Map stores each word once with its count, using merge to add 1. A HashMap has no sorting, so the entries are copied into a List and sorted with a comparator by value, descending, with ties broken alphabetically.

Generic bubbleSort.

```
import java.util.Arrays;

public class BubbleSortDemo {

    // T must be Comparable, so compareTo is guaranteed to exist
    static <T extends Comparable<T>> void bubbleSort(T[] arr) {
        for (int i = 0; i < arr.length - 1; i++) {
            for (int j = 0; j < arr.length - 1 - i; j++) {
                if (arr[j].compareTo(arr[j + 1]) > 0) {
                    // swap the two neighbours
                    T temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                }
            }
```

```
        }
    }

    public static void main(String[] args) {
        Integer[] nums = {5, 2, 9, 1, 7};
        String[] words = {"pear", "apple", "zebra", "mango"};

        bubbleSort(nums);
        bubbleSort(words);

        System.out.println(Arrays.toString(nums));
        System.out.println(Arrays.toString(words));

        // int[] raw = {3, 1, 2};
        // bubbleSort(raw);          // try uncommenting: what error do you get?
    }
}
```

Output:

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab8\exercises> java BubbleSortDemo

[1, 2, 5, 7, 9]

[apple, mango, pear, zebra]

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab8\exercises>

Explanation of why int[] will not work

Generics work only with reference types, and int[] is an array of primitives, so it cannot be a T[]. Java does not convert a whole int[] into an Integer[]. Also, int has no compareTo method. Use Integer[] instead. Include the compile error from trying bubbleSort on an int[].

Conclusion

Generics move type errors from run time to compile time and remove casts. The collection interfaces (List, Set, Map, Queue) let you pick the structure that matches the problem.

Viva answers

1) Why can generics not be used with primitive types, and what is type erasure? Generics work only with reference types, so primitives need wrapper classes such as Integer. After the compiler checks the types, it erases the type parameter, so at run time there is only one version of the class.
2) Difference between Comparable and Comparator? Comparable is the class's one natural order (compareTo). A Comparator is an outside ordering, and there can be many.
3) Why program to the interface (List, not ArrayList)? So the implementation can change without changing the rest of the code.

Lab 9: Nested Types, Enums, Records and the Object Contract

Objectives

1) Tell static nested, inner, local and anonymous classes apart.

2) Model fixed constants with enum, and make immutable data classes with record.
3) Override equals, hashCode and toString consistently.

Sample programs

NestedDemo.java

```
import java.util.Iterator;

public class NestedDemo {

    private final int[] data = {10, 20, 30};
    private static int instances = 0;

    NestedDemo() { instances++; }

    // 1) STATIC NESTED: no outer instance needed
    static class Helper {
        static String describe() { return "Instances created: " + instances; }
    }
```

```
    // 2) INNER CLASS: can read the outer object's private fields
    class DataIterator implements Iterator<Integer> {
        private int index = 0;
        public boolean hasNext() { return index < data.length; }
        public Integer next()    { return data[index++]; }
    }

    public static void main(String[] args) {
        NestedDemo outer = new NestedDemo();
        System.out.println(Helper.describe());

        // EXPLAIN: inner class instance requires an outer instance
        NestedDemo.DataIterator it = outer.new DataIterator();
        while (it.hasNext()) System.out.print(it.next() + " ");
        System.out.println();

        // 3) LOCAL CLASS
        class Greeter {
            String greet(String n) { return "Hello, " + n; }
        }
        System.out.println(new Greeter().greet("class"));

        // 4) ANONYMOUS CLASS: implements Runnable on the spot
        Runnable task = new Runnable() {
            @Override public void run() { System.out.println("Anonymous task
running"); }
        };
        task.run();

        // Modern alternative for single-method interfaces: a lambda (Lab 10)
        Runnable lambdaTask = () -> System.out.println("Lambda task running");
        lambdaTask.run();
    }
}
```

Output

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab9> javac NestedDemo.java

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab9> java NestedDemo

Instances created: 1

10 20 30

Hello, class

Anonymous task running

Lambda task running

EnumDemo.java

```
enum Day { MON, TUE, WED, THU, FRI, SAT, SUN }
// EXPLAIN: an enum can have fields, a private constructor and methods
enum Planet {
MERCURY(3.303e+23, 2.4397e6),
EARTH(5.976e+24, 6.37814e6);
private final double mass; // kg
private final double radius; // m
Planet(double mass, double radius) { this.mass = mass; this.radius = radius; }
static final double G = 6.67300E-11;
double surfaceGravity() { return G * mass / (radius * radius); }
}
// EXPLAIN: each constant may override an abstract method (constant-specific
body)
enum Operation {
ADD("+") { double apply(double a, double b) { return a + b; } },
MULTIPLY("*") { double apply(double a, double b) { return a * b; } };
private final String symbol;
Operation(String symbol) { this.symbol = symbol; }
abstract double apply(double a, double b);
String symbol() { return symbol; }
}
public class EnumDemo {
static String kind(Day d) {
return switch (d) { // EXPLAIN: switch expression; compiler checks all cases

case SAT, SUN -> "Weekend";
default -> "Weekday";
};
}
public static void main(String[] args) {
Day today = Day.WED;
System.out.println(today + " is a " + kind(today) + ", ordinal " +
today.ordinal());
for (Day d : Day.values()) System.out.print(d + " ");
```

```
System.out.println();
System.out.printf("Earth gravity: %.2f m/s^2%n",
Planet.EARTH.surfaceGravity());
System.out.println(Day.valueOf("FRI"));
for (Operation op : Operation.values()) {
System.out.println("6 " + op.symbol() + " 7 = " + op.apply(6, 7));
}
}
}
```

Output:

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab9> javac EnumDemo.java

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab9> java EnumDemo

WED is a Weekday, ordinal 2

MON TUE WED THU FRI SAT SUN

Earth gravity: 9.80 m/s^2

FRI

6 + 7 = 13.0

6 * 7 = 42.0

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab9>

ObjectContract.java

```
import java.util.HashSet;
import java.util.Objects;
import java.util.Set;
class Point {
private final int x, y;
Point(int x, int y) { this.x = x; this.y = y; }
// EXPLAIN: equality by VALUE instead of by reference
@Override
public boolean equals(Object o) {
if (this == o) return true; // same reference
if (!(o instanceof Point other)) return false; // null or wrong type
```

```
return x == other.x && y == other.y;
}
// EXPLAIN: equal objects MUST produce equal hash codes
@Override
public int hashCode() { return Objects.hash(x, y); }
@Override
public String toString() { return "Point(" + x + ", " + y + ")"; }
}
// The same idea with almost no code: a record
record Coordinate(int x, int y) {
// compact constructor: validation
Coordinate {
if (x < 0 || y < 0) throw new IllegalArgumentException("Negative coordinate");

}
double distanceFromOrigin() { return Math.sqrt(x * x + y * y); }
}
public class ObjectContract {
public static void main(String[] args) {
Point p1 = new Point(1, 2);
Point p2 = new Point(1, 2);
System.out.println(p1 == p2); // false (different objects)
System.out.println(p1.equals(p2)); // true (same values)
Set<Point> set = new HashSet<>();
set.add(p1);
set.add(p2); // duplicate by equals/hashCode
System.out.println("Set size: " + set.size()); // 1
System.out.println(p1);
Coordinate c1 = new Coordinate(3, 4);
Coordinate c2 = new Coordinate(3, 4);
System.out.println(c1); // generated toString
System.out.println(c1.equals(c2)); // generated equals
System.out.println(c1.x() + " " + c1.distanceFromOrigin()); // accessor: x()

}
}
```

Output:

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab9> javac ObjectContract.java

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab9> java ObjectContract

false

true

Set size: 1

Point(1, 2)

Coordinate[x=3, y=4]

true

3 5.0

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab9>

Exercises

enum Level with a switch.

```
enum Level { LOW, MEDIUM, HIGH }

public class LevelDemo {

    static void advice(Level level) {
        switch (level) {
            case LOW -> System.out.println("LOW: relax, nothing to do yet");
            case MEDIUM -> System.out.println("MEDIUM: keep an eye on it");
            case HIGH -> System.out.println("HIGH: act now");
        }
    }

    public static void main(String[] args) {
        for (Level l : Level.values()) {
            advice(l);
        }
    }
}
```

Output:

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab9> cd exercises

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab9\exercises> javac *.java

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab9\exercises> java LevelDemo

LOW: relax, nothing to do yet

MEDIUM: keep an eye on it

HIGH: act now

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab9\exercises>

record Student(String name, int age).

```
record Student(String name, int age) { }

public class RecordDemo {
    public static void main(String[] args) {
        Student s1 = new Student("Tariro", 20);
        Student s2 = new Student("Tariro", 20);
        Student s3 = new Student("Farai", 21);

        System.out.println(s1);
        System.out.println("s1 equals s2? " + s1.equals(s2));
        System.out.println("s1 == s2? " + (s1 == s2));
        System.out.println("s1 equals s3? " + s1.equals(s3));
        System.out.println("Name: " + s1.name());
        System.out.println("Age: " + s1.age());
    }
}
```

Output:

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab9\exercises> java RecordDemo

Student[name=Tariro, age=20]

s1 equals s2? true

s1 == s2? false

s1 equals s3? false

Name: Tariro

Age: 20

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab9\exercises>

Explanation

== compares references and equals compares values. A record generates the constructor, accessors, equals, hashCode and toString, and its fields are private and final, so it is immutable.

Book equals and hashCode by ISBN.

```
import java.util.HashSet;
import java.util.Set;

public class BookSetDemo {
    public static void main(String[] args) {
        Set<Book> set = new HashSet<>();

        Book b1 = new Book("978-0132350884", "Clean Code");
        Book b2 = new Book("978-0132350884", "Clean Code (second copy)");
        Book b3 = new Book("978-0134685991", "Effective Java");

        System.out.println("add b1: " + set.add(b1));
        System.out.println("add b2: " + set.add(b2));
        System.out.println("add b3: " + set.add(b3));
        System.out.println("Set size: " + set.size());

        System.out.println("b1.equals(b2)? " + b1.equals(b2));
        System.out.println("b1 == b2? " + (b1 == b2));
        System.out.println("contains lookalike? "
```

```
                + set.contains(new Book("978-0134685991", "anything")));
    }
}
```

Output:

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab9\exercises> java BookSetDemo.java

add b1: true

add b2: false

add b3: true

Set size: 2

b1.equals(b2)? true

b1 == b2? false

contains lookalike? true

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab9\exercises>

Explanation

Without the overrides the output was: add b2: true, Set size: 3, and contains lookalike false. Explanation: by default equals compares references, so two books with the same ISBN count as different. Overriding equals by ISBN and hashCode with the same field keeps the contract that equal objects have equal hash codes. HashSet uses both, so it rejects the duplicate.

LinkedList with a private static nested Node and an inner iterator.

```
public class LinkedListDemo {
    public static void main(String[] args) {
        MyLinkedList list = new MyLinkedList();
        list.add("Tariro");
        list.add("Farai");
        list.add("Rudo");
```

```
        // works because MyLinkedList implements Iterable<String>
        for (String name : list) {
            System.out.println(name);
        }

        System.out.println("Size: " + list.size());

        //MyLinkedList.Node n = null;    // Error: Node has a private class in
MyLinkedList
    }
}
```

Output:

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab9\exercises> java LinkedListDemo

Tariro

Farai

Rudo

Size: 3

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab9\exercises>

Explanation

Node is a private static nested class because it is an internal detail and does not need the outer list. The iterator is an inner class because it must read the list's head. Because the list implements Iterable, the for-each loop works. Error captured: MyLinkedList.Node has private access.

enum Coin with fewestCoins.

```
import java.util.List;

public class CoinDemo {
    public static void main(String[] args) {
        int[] amounts = {67, 30, 99, 0, 5};
```

```
        for (int a : amounts) {
            List<Coin> coins = Coin.fewestCoins(a);
            System.out.println(a + " -> " + coins.size() + " coins: " + coins);
        }
    }
}
```

Output:

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab9\exercises> java CoinDemo

67 -> 6 coins: [QUARTER, QUARTER, DIME, NICKEL, PENNY, PENNY]

30 -> 2 coins: [QUARTER, NICKEL]

99 -> 9 coins: [QUARTER, QUARTER, QUARTER, DIME, DIME, PENNY, PENNY, PENNY, PENNY]

0 -> 0 coins: []

5 -> 1 coins: [NICKEL]

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab9\exercises>

Explanation

Each constant stores its value in a private final field set by a private constructor. fewestCoins is a static method that goes through values() from largest to smallest and takes as many of each coin as fit. This greedy method works for these coin values but not for every coin system (for coins 1, 3 and 4, amount 6, greedy gives 3 coins but 3 + 3 uses 2).

Conclusion

Nested classes keep helper types close to the class that uses them. Enums and records give type-safe constants and compact immutable data, and equals and hashCode must always be overridden together.

Viva answers

1) Why must hashCode be overridden whenever equals is overridden? Equal objects must have equal hash codes. Otherwise hash-based collections such as HashSet and HashMap look in the wrong bucket and misbehave.
2) Static nested versus inner class? A static nested class does not need an outer instance. An inner class holds a hidden reference to an outer object and can use its fields.
3) Why is equals(Object o) written with an Object parameter? With any other parameter type it only overloads equals and does not override it.

Lab 10: File I/O, Lambdas and Streams

Objectives

1) Read and write text files with java.nio.file and handle IOException.
2) Write lambdas and method references, and identify functional interfaces.
3) Process collections with the Stream API.

Sample programs

FileDemo: writes and reads students.txt, and handles a missing file.

```
import java.io.IOException;
import java.nio.file.*;
import java.util.List;
public class FileDemo {
public static void main(String[] args) {
Path path = Path.of("students.txt");
// ---- WRITE ----
List<String> lines = List.of("Tariro,101,3.8", "Farai,102,3.1", "Rudo,103,3.9");

try {
Files.write(path, lines); // creates or overwrites
Files.writeString(path, "Kuda,104,3.2\n", StandardOpenOption.APPEND);
System.out.println("File written: " + Files.size(path) + " bytes");
} catch (IOException e) {
System.out.println("Write failed: " + e.getMessage());
return;
}
// ---- READ (try-with-resources, streams a large file line by line) ----
try (var reader = Files.newBufferedReader(path)) {
String line;
int count = 0;
while ((line = reader.readLine()) != null) {
```

```
String[] parts = line.split(",");
System.out.printf("%-8s roll=%s gpa=%s%n", parts[0], parts[1],
parts[2]);
count++;
}
System.out.println(count + " records read");
} catch (IOException e) {
System.out.println("Read failed: " + e.getMessage());
}
// ---- HANDLING A MISSING FILE ----
try {
Files.readAllLines(Path.of("does-not-exist.txt"));
} catch (NoSuchFileException e) {
System.out.println("Missing file: " + e.getMessage());
} catch (IOException e) {
System.out.println("I/O error");
}
}
}
```

Output:

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab10> javac FileDemo.java

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab10> java FileDemo

File written: 58 bytes

Tariro roll=101 gpa=3.8

Farai roll=102 gpa=3.1

Rudo roll=103 gpa=3.9

Kuda roll=104 gpa=3.2

4 records read

Missing file: does-not-exist.txt

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab10>

StudentRepository: saves and loads Learner records as CSV.

```
import java.io.IOException;
import java.nio.file.*;
import java.util.*;
record Learner(String name, int roll, double gpa) {
String toCsv() { return name + "," + roll + "," + gpa; }
// EXPLAIN: a static factory that parses one CSV line back into an object
static Learner fromCsv(String line) {
String[] p = line.split(",");
return new Learner(p[0], Integer.parseInt(p[1]), Double.parseDouble(p[2]));
}
}
public class StudentRepository {
private final Path file;
public StudentRepository(String fileName) { this.file = Path.of(fileName); }
public void saveAll(List<Learner> learners) throws IOException {
List<String> lines = new ArrayList<>();
for (Learner l : learners) lines.add(l.toCsv());
Files.write(file, lines);
}
public List<Learner> loadAll() throws IOException {
List<Learner> result = new ArrayList<>();
if (!Files.exists(file)) return result; // first run: nothing saved yet

for (String line : Files.readAllLines(file)) {
if (!line.isBlank()) result.add(Learner.fromCsv(line));
}
return result;
}
public static void main(String[] args) throws IOException {
StudentRepository repo = new StudentRepository("learners.csv");
repo.saveAll(List.of(new Learner("Ada", 1, 3.9), new Learner("Kuda", 2,
3.2)));
for (Learner l : repo.loadAll()) System.out.println(l);
}
}
```

Output:

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab10> javac StudentRepository.java

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab10> java StudentRepository

Learner[name=Ada, roll=1, gpa=3.9]

Learner[name=Kuda, roll=2, gpa=3.2]

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab10>

StreamDemo: filter, map, sorted, reduce and groupingBy on a list of employees.

```
import java.util.*;
import java.util.function.*;
import java.util.stream.*;
record Employee(String name, String dept, double salary) {}
public class StreamDemo {
public static void main(String[] args) {
// ---- Lambdas and functional interfaces ----
Predicate<String> longWord = s -> s.length() > 4;
Function<String, Integer> length = String::length; // method reference
Consumer<String> shout = s -> System.out.println(s.toUpperCase() + "!");
Supplier<List<String>> maker = ArrayList::new;
System.out.println(longWord.test("lambda") + " " + length.apply("stream"));
shout.accept("hello");
List<String> empty = maker.get();
// ---- Streams over a list ----
List<Employee> staff = List.of(
new Employee("Ada", "IT", 4200),
new Employee("Ben", "IT", 3800),
new Employee("Cara", "HR", 3000),
new Employee("Dan", "Finance", 5000),
new Employee("Eve", "HR", 3400));
// filter -> map -> collect
List<String> highEarners = staff.stream()
.filter(e -> e.salary() >= 3800)
.map(Employee::name)
.collect(Collectors.toList());
System.out.println("High earners: " + highEarners);
// sorted with a Comparator, limit, forEach
staff.stream()
.sorted(Comparator.comparingDouble(Employee::salary).reversed())
.limit(2)
```

```
.forEach(e -> System.out.println("Top: " + e.name()));
// reduce / numeric summaries
double total = staff.stream().mapToDouble(Employee::salary).sum();
double average =
staff.stream().mapToDouble(Employee::salary).average().orElse(0);
System.out.printf("Total %.0f, average %.1f%n", total, average);
// groupingBy: a Map from department to employees / to a statistic
Map<String, Long> countByDept = staff.stream()
.collect(Collectors.groupingBy(Employee::dept, TreeMap::new,
Collectors.counting()));
System.out.println("Head count: " + countByDept);
Map<String, Double> avgByDept = staff.stream()
.collect(Collectors.groupingBy(Employee::dept, TreeMap::new,
Collectors.averagingDouble(Employee::salary)));
System.out.println("Average pay: " + avgByDept);
// anyMatch / joining
System.out.println("Anyone in Finance? " + staff.stream().anyMatch(e ->
e.dept().equals("Finance")));
System.out.println(staff.stream().map(Employee::name).collect(Collectors.joining(
",", "[", "]")));

}
}
```

Output:

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab10> javac StreamDemo.java

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab10> java StreamDemo

true 6

HELLO!

High earners: [Ada, Ben, Dan]

Top: Dan

Top: Ada

Total 19400, average 3880.0

Head count: {Finance=1, HR=2, IT=2}

Average pay: {Finance=5000.0, HR=3200.0, IT=4000.0}

Anyone in Finance? true

[Ada,Ben,Cara,Dan,Eve]

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab10>

Exercises

FileStats.java

```java
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.NoSuchFileException;
import java.nio.file.Path;
import java.util.List;

public class FileStats {
    public static void main(String[] args) {
        Path path = Path.of("sample.txt");

        try {
            List<String> lines = Files.readAllLines(path);

            int lineCount = lines.size();
            int wordCount = 0;
            int charCount = 0;

            for (String line : lines) {
                charCount += line.length();
                if (!line.isBlank()) {
                    wordCount += line.trim().split("\\s+").length;
                }
            }

            System.out.println("File: " + path);
            System.out.println("Lines: " + lineCount);
            System.out.println("Words: " + wordCount);
            System.out.println("Characters: " + charCount);
```

```
        } catch (NoSuchFileException e) {
            System.out.println("File not found: " + e.getMessage());
        } catch (IOException e) {
            System.out.println("Could not read the file: " + e.getMessage());
        }
    }
}
```

Output

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab10\exercises> javac *.java

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab10\exercises> java FileStats

File: sample.txt

Lines: 3

Words: 8

Characters: 37

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab10\exercises>

RemoveEvens.java

```
import java.util.ArrayList;
import java.util.List;
import java.util.function.Predicate;

public class RemoveEvens {
    public static void main(String[] args) {
        // new ArrayList<>(...) makes the list changeable; List.of alone is
immutable
        List<Integer> nums = new ArrayList<>(List.of(1, 2, 3, 4, 5, 6, 7, 8, 9,
10));

        Predicate<Integer> isEven = n -> n % 2 == 0;
```

```
        nums.removeIf(isEven);

        System.out.println(nums);
    }
}
```

Output:

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab10\exercises> java RemoveEvens

[1, 3, 5, 7, 9]

Explanation

A Predicate is a functional interface with one method, test, that returns a boolean. The lambda n -> n % 2 == 0 supplies it, and removeIf removes every element for which it is true. The list must be changeable, because List.of is immutable.

SaveAndRead.java

```
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

public class SaveAndRead {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        List<String> lines = new ArrayList<>();

        for (int i = 1; i <= 5; i++) {
            System.out.print("Line " + i + ": ");
            lines.add(in.nextLine());
        }

        Path path = Path.of("notes.txt");

        try {
            Files.write(path, lines);                    // save all lines
            System.out.println("Saved to " + path.toAbsolutePath());
```

```
            List<String> back = Files.readAllLines(path);          // read all
lines back
            System.out.println("--- read back ---");
            for (String line : back) {
                System.out.println(line);
            }
        } catch (IOException e) {
            System.out.println("File problem: " + e.getMessage());
        }

        in.close();
    }
}
```

Output

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab10\exercises> java SaveAndRead

Line 1: Hello Java

Line 2: Files are fun

Line 3: Harare is home

Line 4: Coding is fun

Line 5: Debugging is boring

Saved to C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab10\exercises\notes.txt

--- read back ---

Hello Java

Files are fun

Harare is home

Coding is fun

Debugging is boring

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab10\exercises>

Aboveaverage.java

```java
import java.util.List;

record Student(String name, int marks) { }

public class AboveAverage {
    public static void main(String[] args) {
        List<Student> students = List.of(
            new Student("Tariro", 72),
            new Student("Farai", 58),
            new Student("Rudo", 85),
            new Student("Kuda", 64),
            new Student("Ada", 91),
            new Student("Blessing", 49),
            new Student("Lynn", 77)
        );

        // pass 1: the class average
        double average = students.stream()
            .mapToInt(Student::marks)
            .average()
            .orElse(0);

        System.out.printf("Class average: %.2f%n", average);

        // pass 2: filter, map, sort, print
        students.stream()
            .filter(s -> s.marks() > average)
            .map(Student::name)
            .sorted()
            .forEach(System.out::println);
    }
}
```

Output:

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab10\exercises> java AboveAverage

Class average: 70.86

Ada

Lynn

Rudo

Tariro

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab10\exercises>

Explanation

A stream is a pipeline that does not change the source list. mapToInt(...).average() gives the average, then filter keeps students above it, map extracts the names and sorted() orders them. Nothing runs until the terminal operation forEach, so streams are lazy.

Group learners by grade band and write one report per band. With A at GPA 3.5 or more, B at 3.0 or more, and C below 3.0:

```
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import java.util.TreeMap;
import java.util.stream.Collectors;

record Learner(String name, int roll, double gpa) {

    // parse one CSV line into a Learner
    static Learner fromCsv(String line) {
        String[] p = line.split(",");
        return new Learner(p[0], Integer.parseInt(p[1]),
Double.parseDouble(p[2]));
    }
}

public class ReportByBand {
```

```java
    // my grade bands: change the cut-offs if your lecturer gave a scale
    static String band(double gpa) {
        if (gpa >= 3.5) {
            return "A";
        } else if (gpa >= 3.0) {
            return "B";
        }
        return "C";
    }

    public static void main(String[] args) {
        Path input = Path.of("learners.csv");

        try {
            List<Learner> learners = Files.readAllLines(input).stream()
                .filter(line -> !line.isBlank())
                .map(Learner::fromCsv)
                .collect(Collectors.toList());

            // group: band letter -> list of learners (TreeMap keeps A, B, C in
order)
            Map<String, List<Learner>> byBand = learners.stream()
                .collect(Collectors.groupingBy(l -> band(l.gpa()), TreeMap::new,
Collectors.toList()));

            for (Map.Entry<String, List<Learner>> entry : byBand.entrySet()) {
                List<String> lines = new ArrayList<>();
                lines.add("Band " + entry.getKey() + " report (" +
entry.getValue().size() + " learners)");
                for (Learner l : entry.getValue()) {
                    lines.add(l.name() + ", roll " + l.roll() + ", GPA " +
l.gpa());
                }

                Path out = Path.of("report-" + entry.getKey() + ".txt");
                Files.write(out, lines);
                System.out.println("Wrote " + out + ": " +
entry.getValue().size() + " learners");
            }

        } catch (IOException e) {
            System.out.println("File problem: " + e.getMessage());
        }
    }
}
```

Output:

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab10\exercises> java ReportByBand

Wrote report-A.txt: 3 learners

Wrote report-B.txt: 3 learners

Wrote report-C.txt: 2 learners

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab10\exercises>

Explanation

The CSV lines become Learner records, Collectors.groupingBy uses the band method to build a Map from band to learners, and a loop writes one file per band with Files.write.

Conclusion

File methods let a program keep data between runs, but every file operation can fail, so IOException must be handled. Lambdas and streams let you describe what to do with data instead of writing loops by hand.

Viva answers

1) What is a functional interface, and how does a lambda relate to it? A functional interface has exactly one abstract method (Predicate, Function, Comparator, Runnable). A lambda is a short way to supply that one method.
2) Why do most file methods throw IOException? File operations can fail for reasons outside the program's control, and IOException is a checked exception, so it must be caught or declared.
3) What is the difference between an intermediate and a terminal stream operation? Intermediate operations (filter, map, sorted) are lazy and return a stream. A terminal operation (collect, forEach, count) starts the work.
4) Why can a stream not be reused? After a terminal operation it is closed, and reusing it throws IllegalStateException.